



The OpenCLTM SPIR-V Environment Specification

Khronos[®] OpenCL Working Group

Version v3.1.2, Thu, 17 Sep 2026 12:00:00 +0000: from git branch: main commit:
219be24fc8947c68ee45b4d25b55a92b8f799f29

Table of Contents

1. Introduction	2
2. Common Properties	3
2.1. Supported SPIR-V Versions	3
2.2. Extended Instruction Sets	3
2.3. Source Language Encoding	3
2.4. Numerical Type Formats	4
2.5. Supported Types	4
2.6. Image Channel Order Mapping	5
2.7. Image Channel Data Type Mapping	6
2.8. Kernels	7
2.9. Built-in Variables	8
2.10. Alignment of Types	9
2.11. Printf Operands	9
3. Required Capabilities	11
3.1. SPIR-V 1.0	11
3.2. SPIR-V 1.1	12
3.3. SPIR-V 1.2	12
3.4. SPIR-V 1.3	12
3.5. SPIR-V 1.4	13
3.6. SPIR-V 1.5	13
3.7. SPIR-V 1.6	13
3.8. SPIR-V Extensions	13
4. Validation Rules	15
5. OpenCL Extensions	18
5.1. Declaring SPIR-V Extensions	18
5.2. Full and Embedded Profile Extensions	18
5.3. Embedded Profile Extensions	27
6. OpenCL Numerical Compliance	28
6.1. Rounding Modes	28
6.2. Rounding Modes for Conversions	28
6.3. Out-of-Range Conversions	29
6.4. INF, NaN, and Denormalized Numbers	29
6.5. Floating-Point Exceptions	29
6.6. Relative Error as ULPs	30
6.7. Edge Case Behavior	43
7. Image Addressing and Filtering	53
7.1. Image Coordinates	53
7.2. Addressing and Filter Modes	53

7.3. Precision of Addressing and Filter Modes	60
7.4. Conversion Rules	60
7.5. Selecting an Image from an Image Array	72
7.6. Coordinate Format for Reading and Writing Images	72
7.7. Data Format for Reading and Writing Images	74
7.8. Sampled and Sampler-less Reads	76
8. Normative References	77
Appendix A: Changes to OpenCL	78
Summary of changes to OpenCL 3.0	78
Summary of Changes from OpenCL 3.0 to OpenCL 3.1	79

Copyright 2008-2026 The Khronos Group Inc.

This Specification is protected by copyright laws and contains material proprietary to Khronos. Except as described by these terms, it or any components may not be reproduced, republished, distributed, transmitted, displayed, broadcast or otherwise exploited in any manner without the express prior written permission of Khronos.

This Specification has been created under the Khronos Intellectual Property Rights Policy, which is Attachment A of the Khronos Group Membership Agreement available at www.khronos.org/files/member_agreement.pdf and defines the terms 'Scope', 'Compliant Portion', and 'Necessary Patent Claims'.

Khronos grants a conditional copyright license to use and reproduce the unmodified Specification for any purpose, without fee or royalty, EXCEPT no licenses to any patent, trademark or other intellectual property rights are granted under these terms. Parties desiring to implement the Specification and make use of Khronos trademarks in relation to that implementation, and receive reciprocal patent license protection under the Khronos Intellectual Property Rights Policy must become Adopters and confirm the implementation as conformant under the process defined by Khronos for this Specification; see <https://www.khronos.org/adopters>.

Khronos makes no, and expressly disclaims any, representations or warranties, express or implied, regarding this Specification, including, without limitation: merchantability, fitness for a particular purpose, non-infringement of any intellectual property, correctness, accuracy, completeness, timeliness, and reliability. Under no circumstances will Khronos, or any of its Promoters, Contributors or Members, or their respective partners, officers, directors, employees, agents or representatives be liable for any damages, whether direct, indirect, special or consequential damages for lost revenues, lost profits, or otherwise, arising from or in connection with these materials.

Where this Specification identifies specific sections of external references, only those specifically identified sections define normative functionality. The Khronos Intellectual Property Rights Policy excludes external references to materials and associated enabling technology not created by Khronos from the Scope of this specification, and any licenses that may be required to implement such referenced materials and associated technologies must be obtained separately and may involve royalty payments.

Khronos® and Vulkan® are registered trademarks, and SPIR™, SPIR-V™, and SYCL™ are trademarks of The Khronos Group Inc. OpenCL™ is a trademark of Apple Inc. used under license by Khronos. OpenGL® is a registered trademark and the OpenGL ES™ and OpenGL SC™ logos are trademarks of Hewlett Packard Enterprise used under license by Khronos. All other product names, trademarks, and/or company names are used solely for identification and belong to their respective owners.

Chapter 1. Introduction

[OpenCL](#) is an open, royalty-free, standard for general purpose parallel programming across CPUs, GPUs, and other processors, giving software developers portable and efficient access to the power of heterogeneous processing platforms.

[SPIR-V](#) is an open, royalty-free, standard intermediate language capable of representing parallel compute kernels that are executed by implementations of the OpenCL standard.

SPIR-V is adaptable to multiple execution environments: a SPIR-V module is consumed by an execution environment, as specified by a client API. This document describes the SPIR-V execution environment for implementations of the OpenCL standard. The SPIR-V execution environment describes required support for some SPIR-V capabilities, additional semantics for some SPIR-V instructions, and additional validation rules that a SPIR-V binary module must adhere to in order to be considered valid.

This document is written for compiler developers who are generating SPIR-V modules intended to be consumed by the OpenCL API, for implementors of the OpenCL API who are consuming SPIR-V modules, and for software developers who are using SPIR-V modules with the OpenCL API.

Chapter 2. Common Properties

This section describes common properties of all OpenCL environments that consume SPIR-V modules.

A SPIR-V module passed to an OpenCL environment is interpreted as a series of 32-bit words in host endianness, with literal strings packed as described in the SPIR-V specification. The first few words of the SPIR-V module must be a magic number and a SPIR-V version number, as described in the SPIR-V specification.

2.1. Supported SPIR-V Versions

An OpenCL device describes the versions of SPIR-V modules that it supports using the `CL_DEVICE_IL_VERSION` query in OpenCL 2.1 or newer, the `CL_DEVICE_ILS_WITH_VERSION` query in OpenCL 3.0 or newer, or the `CL_DEVICE_IL_VERSION_KHR` query in the `cl_khr_il_program` extension.

OpenCL devices that support the `cl_khr_il_program` extension or OpenCL 2.1 must support SPIR-V 1.0 modules. OpenCL devices that support OpenCL 2.2 must support SPIR-V 1.0, 1.1, and 1.2 modules. There is no requirement for an OpenCL 3.0 device to support SPIR-V. OpenCL devices that support OpenCL 3.1 must support SPIR-V 1.0, 1.1, 1.2, 1.3, and 1.4 modules.

2.2. Extended Instruction Sets

OpenCL environments supporting SPIR-V must support SPIR-V modules that import the `OpenCL.std` [extended instruction set for OpenCL](#) using `OpExtInstImport`. For example:

```
... = OpExtInstImport "OpenCL.std"
```

2.3. Source Language Encoding

If a SPIR-V module represents a program written in OpenCL C, then the *Source Language* operand for the `OpSource` instruction should be `OpenCL_C`, and the 32-bit literal language *Version* should describe the version of OpenCL C, encoded MSB to LSB as:

```
0 | Major Number | Minor Number | Revision Number (optional)
```

If a SPIR-V module represents a program written in OpenCL C++, then the *Source Language* operand for the `OpSource` instruction should be `OpenCL_CPP`, and the 32-bit literal language *Version* should describe the version of OpenCL C++, encoded similarly.

The source language version is purely informational and has no semantic meaning.

2.4. Numerical Type Formats

For all OpenCL environments, floating-point types are represented and stored using [IEEE-754](#) semantics. All integer formats are represented and stored using 2's-complement format.

2.5. Supported Types

The following types are supported by OpenCL environments. Note that some types may require additional capabilities, and may not be supported by all OpenCL environments.

OpenCL environments support arrays declared using **OpTypeArray**, structs declared using **OpTypeStruct**, functions declared using **OpTypeFunction**, and pointers declared using **OpTypePointer**.

2.5.1. Basic Scalar and Vector Types

OpTypeVoid is supported.

The following scalar types are supported by OpenCL environments:

- **OpTypeBool**
- **OpTypeInt**, with *Width* equal to 8, 16, 32, or 64, and with *Signedness* equal to zero, indicating no signedness semantics.
- **OpTypeFloat**, with *Width* equal to 16, 32, or 64.

OpenCL environments support vector types declared using **OpTypeVector**. The vector *Component Type* may be any of the scalar types described above. Supported vector *Component Counts* are 2, 3, 4, 8, or 16.

2.5.2. Image-Related Data Types

The following table describes the **OpTypeImage** image types supported by OpenCL environments:

Table 1. Image Types

<i>Dim</i>	<i>Depth</i>	<i>Arrayed</i>	Description
1D	0	0	A 1D image.
1D	0	1	A 1D image array.
2D	0	0	A 2D image.
2D	1	0	A 2D depth image.
2D	0	1	A 2D image array.
2D	1	1	A 2D depth image array.
3D	0	0	A 3D image.
Buffer	0	0	A 1D buffer image.

OpTypeSampler may be used to declare sampler types in OpenCL environments.

OpTypeSampledImage may be used to declare combined image and sampler types in OpenCL environments.

2.5.3. Other Data Types

The following table describes other data types that may be used in an OpenCL environment:

Table 2. Other Data Types

Type	Description
OpTypeEvent	OpenCL event representing async copies from global to local memory and vice-versa.
OpTypeDeviceEvent	OpenCL device-side event representing commands enqueued to device command-queues.
OpTypePipe	OpenCL pipe.
OpTypeReserveId	OpenCL pipe reservation identifier.
OpTypeQueue	OpenCL device-side command-queue.

2.6. Image Channel Order Mapping

The following table describes how the results of the SPIR-V **OpImageQueryOrder** instruction correspond to the OpenCL host API image channel orders.

Table 3. Image Channel Order mapping

SPIR-V Image Channel Order		OpenCL Image Channel Order
0	R	CL_R
1	A	CL_A
2	RG	CL_RG
3	RA	CL_RA
4	RGB	CL_RGB
5	RGBA	CL_RGBA
6	BGRA	CL_BGRA
7	ARGB	CL_ARGB
8	Intensity	CL_INTENSITY
9	Luminance	CL_LUMINANCE
10	Rx	CL_Rx
11	RGx	CL_RGx
12	RGBx	CL_RGBx

SPIR-V Image Channel Order		OpenCL Image Channel Order
13	Depth	CL_DEPTH
14	DepthStencil	CL_DEPTH_STENCIL
15	sRGB	CL_sRGB
16	sRGBx	CL_sRGBx
17	sRGBA	CL_sRGBA
18	sBGRA	CL_sBGRA
19	ABGR	CL_ABGR



The SPIR-V Image Channel Orders are enumerated in the same order as the OpenCL Channel Order enums to enable simple conversion between the two.

2.7. Image Channel Data Type Mapping

The following table describes how the results of the SPIR-V **OpImageQueryFormat** instruction correspond to the OpenCL host API image channel data types.

Table 4. Image Channel Data Type mapping

SPIR-V Image Channel Data Type		OpenCL Image Channel Data Type
0	SnormInt8	CL_SNORM_INT8
1	SnormInt16	CL_SNORM_INT16
2	UnormInt8	CL_UNORM_INT8
3	UnormInt16	CL_UNORM_INT16
4	UnormShort565	CL_UNORM_SHORT_565
5	UnormShort555	CL_UNORM_SHORT_555
6	UnormInt101010	CL_UNORM_INT_101010
7	SignedInt8	CL_SIGNED_INT8
8	SignedInt16	CL_SIGNED_INT16
9	SignedInt32	CL_SIGNED_INT32
10	UnsignedInt8	CL_UNSIGNED_INT8
11	UnsignedInt16	CL_UNSIGNED_INT16
12	UnsignedInt32	CL_UNSIGNED_INT32
13	HalfFloat	CL_HALF_FLOAT
14	Float	CL_FLOAT
15	UnormInt24	CL_UNORM_INT24
16	UnormInt101010_2	CL_UNORM_INT_101010_2
17	UnormInt10X6EXT	CL_UNORM_INT10X6_EXT

SPIR-V Image Channel Data Type		OpenCL Image Channel Data Type
21	UnormInt2_101010EXT	CL_UNORM_INT_2_101010_EXT
22	UnsignedInt10X6EXT	CL_UNSIGNED_INT10X6_EXT
23	UnsignedInt12X4EXT	CL_UNSIGNED_INT12X4_EXT
24	UnsignedInt14X2EXT	CL_UNSIGNED_INT14X2_EXT
25	UnormInt12X4EXT	CL_UNORM_INT12X4_EXT
26	UnormInt14X2EXT	CL_UNORM_INT14X2_EXT



The SPIR-V Image Channel Data Types are enumerated in the same order as the OpenCL Channel Data Type enums to enable simple conversion between the two.

2.8. Kernels

An **OpFunction** in a SPIR-V module that is identified with **OpEntryPoint** defines an OpenCL kernel that may be invoked using the OpenCL host API enqueue kernel interfaces.

2.8.1. Kernel Return Types

The *Result Type* for an **OpFunction** identified with **OpEntryPoint** must be **OpTypeVoid**.

2.8.2. Kernel Arguments

An **OpFunctionParameter** for an **OpFunction** that is identified with **OpEntryPoint** defines an OpenCL kernel argument. Allowed types for OpenCL kernel arguments are:

- **OpTypeInt**
- **OpTypeFloat**
- **OpTypeStruct**
- **OpTypeVector**
- **OpTypePointer**
- **OpTypeSampler**
- **OpTypeImage**
- **OpTypePipe**
- **OpTypeQueue**

For **OpTypeInt** parameters, supported *Widths* are 8, 16, 32, and 64, and must have no signedness semantics.

For **OpTypeFloat** parameters, supported *Width* are 16, 32, and 64.

For **OpTypeStruct** parameters, supported structure *Member Types* are:

- **OpTypeInt**

- **OpTypeFloat**
- **OpTypeStruct**
- **OpTypeVector**
- **OpTypePointer**

For **OpTypePointer** parameters, supported *Storage Classes* are:

- **CrossWorkgroup**
- **Workgroup**
- **UniformConstant**

OpenCL kernel argument types must have a representation in the OpenCL host API.

Environments that support extensions or optional features may allow additional types in an entry point's parameter list.

2.9. Built-in Variables

An **OpVariable** in a SPIR-V module with the **BuiltIn** decoration represents a built-in variable. All built-in variables must be in the **Input** storage class.

The following table describes the required SPIR-V type for built-in variables. In this table, **size_t** is used as a generic type to represent:

- **OpTypeInt** with *Width* equal to 32 if the *Addressing Model* declared in **OpMemoryModel** is **Physical32**.
- **OpTypeInt** with *Width* equal to 64 if the *Addressing Model* declared in **OpMemoryModel** is **Physical64**.

The mapping from an OpenCL C built-in function to the SPIR-V **BuiltIn** is informational and non-normative.

OpenCL C Function	SPIR-V BuiltIn	Required SPIR-V Type
get_work_dim	WorkDim	OpTypeInt with <i>Width</i> equal to 32
get_global_size	GlobalSize	OpTypeVector of 3 components of size_t
get_global_id	GlobalInvocationId	OpTypeVector of 3 components of size_t
get_local_size	WorkgroupSize	OpTypeVector of 3 components of size_t
get_enqueued_local_size	EnqueuedWorkgroupSize	OpTypeVector of 3 components of size_t
get_local_id	LocalInvocationId	OpTypeVector of 3 components of size_t
get_num_groups	NumWorkgroups	OpTypeVector of 3 components of size_t
get_group_id	WorkgroupId	OpTypeVector of 3 components of size_t
get_global_offset	GlobalOffset	OpTypeVector of 3 components of size_t
get_global_linear_id	GlobalLinearId	size_t

OpenCL C Function	SPIR-V BuiltIn	Required SPIR-V Type
<code>get_local_linear_id</code>	LocalInvocationIndex	<code>size_t</code>
<code>get_sub_group_size</code>	SubgroupSize	OpTypeInt with <i>Width</i> equal to 32
<code>get_max_sub_group_size</code>	SubgroupMaxSize	OpTypeInt with <i>Width</i> equal to 32
<code>get_num_sub_groups</code>	NumSubgroups	OpTypeInt with <i>Width</i> equal to 32
<code>get_enqueued_num_sub_group_s</code>	NumEnqueuedSubgroups	OpTypeInt with <i>Width</i> equal to 32
<code>get_sub_group_id</code>	SubgroupId	OpTypeInt with <i>Width</i> equal to 32
<code>get_sub_group_local_id</code>	SubgroupLocalInvocationId	OpTypeInt with <i>Width</i> equal to 32

2.10. Alignment of Types

Objects of type **OpTypeInt**, **OpTypeFloat**, and **OpTypePointer** must be aligned in memory to the size of the type in bytes. Objects of type **OpTypeVector** with these component types must be aligned in memory to the size of the vector type in bytes. For 3-component vector types, the size of the vector type is four times the size the component type.

The compiler is responsible for aligning objects allocated by **OpVariable** to the appropriate alignment as required by the *Result Type*.

For **OpTypePointer** arguments to a function, the compiler may assume that the pointer is appropriately aligned as required by the *Type* that the pointer points to.

Behavior of an unaligned load or store is undefined.

2.11. Printf Operands

For the **printf** instruction in the **OpenCL.std** [extended instruction set for OpenCL](#), the format specifiers in the *format* operand determine how to print the additional argument operands.

Behavior is undefined unless the source operand type for an additional argument matches the supported operand types for the format specifier in the following table.

In this table, `size_t` and `ptrdiff_t` are used as generic types to represent:

- **OpTypeInt** with *Width* equal to 32 if the *Addressing Model* declared in **OpMemoryModel** is **Physical32**.
- **OpTypeInt** with *Width* equal to 64 if the *Addressing Model* declared in **OpMemoryModel** is **Physical64**.

Vector Specifier	Length Modifier	Conversion Specifier	Supported Operand Type
(none)	hh, h , (none)	d, i, o, u, x, X	OpTypeInt with <i>Width</i> equal to 32

Vector Specifier	Length Modifier	Conversion Specifier	Supported Operand Type
(none)	l	d, i, o, u, x, X	OpTypeInt with <i>Width</i> equal to 64
(none)	(none)	a, A, e, E, f, F, g, G	OpTypeFloat with <i>Width</i> equal to 32 ^[1] , OpTypeFloat with <i>Width</i> equal to 64
vn	hh	d, i, o, u, x, X	OpTypeVector with <i>n</i> components of OpTypeInt with <i>Width</i> equal to 8
vn	h	d, i, o, u, x, X	OpTypeVector with <i>n</i> components of OpTypeInt with <i>Width</i> equal to 16
vn	hl	d, i, o, u, x, X	OpTypeVector with <i>n</i> components of OpTypeInt with <i>Width</i> equal to 32
vn	l	d, i, o, u, x, X	OpTypeVector with <i>n</i> components of OpTypeInt with <i>Width</i> equal to 64
vn	h	a, A, e, E, f, F, g, G	OpTypeVector with <i>n</i> components of OpTypeFloat with <i>Width</i> equal to 16
vn	hl	a, A, e, E, f, F, g, G	OpTypeVector with <i>n</i> components of OpTypeFloat with <i>Width</i> equal to 32
vn	l	a, A, e, E, f, F, g, G	OpTypeVector with <i>n</i> components of OpTypeFloat with <i>Width</i> equal to 64
For devices supporting OpenCL 3.1 or newer:			
(none)	z	d, i, o, u, x, X	size_t
(none)	t	d, i, o, u, x, X	ptrdiff_t

The following additional restrictions also apply to the **printf** instruction in the **OpenCL.std** extended instruction set for OpenCL:

- Behavior is undefined for the asterisk ***** field width modifier.
- Behavior is undefined when the **l** length modifier is followed by the **c** conversion specifier or the **s** conversion specifier.
- Behavior is undefined for the **ll**, **j**, and **L** length modifiers. Prior to OpenCL 3.1, behavior is additionally undefined for the **z** and **t** length modifiers.
- Behavior is undefined for the **n** conversion specifier.
- Behavior is undefined for the **l** length modifier when 64-bit integers are not supported.
- Behavior is undefined for the conversion specifier **s** when the operand is not a literal string.

[1] This allows printing 16-bit **half** and 32-bit **float** values on devices that both do and do not support the **Float64** capability for double-precision floating-point.

Chapter 3. Required Capabilities

3.1. SPIR-V 1.0

An OpenCL environment that supports SPIR-V 1.0 must support SPIR-V 1.0 modules that declare the following capabilities:

- **Addresses**
- **Float16Buffer**
- **Int64**
 - For Full Profile devices.
- **Int16**
- **Int8**
- **Kernel**
- **Linkage**
- **Vector16**
- **DeviceEnqueue**
 - For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Device-Side Enqueue (where `CL_DEVICE_DEVICE_ENQUEUE_CAPABILITIES` is not `0`).
- **GenericPointer**
 - For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting the Generic Address Space (where `CL_DEVICE_GENERIC_ADDRESS_SPACE_SUPPORT` is `CL_TRUE`).
- **Groups**
 - For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not `0`) or Work-group Collective Functions (where `CL_DEVICE_WORK_GROUP_COLLECTIVE_FUNCTIONS_SUPPORT` is `CL_TRUE`).
- **Pipes**
 - For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Pipes (where `CL_DEVICE_PIPE_SUPPORT` is `CL_TRUE`).
- **ImageBasic**
 - For devices supporting Images (where `CL_DEVICE_IMAGE_SUPPORT` is `CL_TRUE`)
- **Float64**
 - For devices supporting Double Precision Floating-Point (where `CL_DEVICE_DOUBLE_FP_CONFIG` is not `0`)

If the OpenCL environment supports the **ImageBasic** capability, then the following capabilities must also be supported:

- **LiteralSampler**

- **Sampled1D**
- **Image1D**
- **SampledBuffer**
- **ImageBuffer**
- **ImageReadWrite**
 - For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Read-Write Images (where `CL_DEVICE_MAX_READ_WRITE_IMAGE_ARGS` is not 0)

3.2. SPIR-V 1.1

An OpenCL environment supporting SPIR-V 1.1 must support SPIR-V 1.1 modules that declare the capabilities required for SPIR-V 1.0 modules, above.

In addition, an OpenCL environment consuming SPIR-V 1.1 must support SPIR-V 1.1 modules that declare the following capabilities:

- **SubgroupDispatch**
 - For OpenCL 2.2 devices, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not 0)
- **PipeStorage**
 - For OpenCL 2.2 devices.

3.3. SPIR-V 1.2

An OpenCL environment supporting SPIR-V 1.2 must support SPIR-V 1.2 modules that declare the capabilities required for SPIR-V 1.1 modules, above.

SPIR-V 1.2 does not add any new required capabilities.

3.4. SPIR-V 1.3

An OpenCL environment supporting SPIR-V 1.3 must support SPIR-V 1.3 modules that declare the capabilities required for SPIR-V 1.2 modules, above.

Prior to OpenCL 3.1, SPIR-V 1.3 did not add any new required capabilities.

An OpenCL environment supporting SPIR-V 1.3 and OpenCL 3.1 must support SPIR-V 1.3 modules that declare the following capabilities:

- **GroupNonUniformShuffle**
- **GroupNonUniformShuffleRelative**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

For the instructions **OpGroupNonUniformShuffle** and **OpGroupNonUniformShuffleXor** requiring these capabilities, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to `char`, `uchar`, `short`, `ushort`, `int`, `uint`, `long`, and `ulong`)
 - **OpTypeFloat** (equivalent to `half`, `float`, and `double`)

3.5. SPIR-V 1.4

An OpenCL environment supporting SPIR-V 1.4 must support SPIR-V 1.4 modules that declare the capabilities required for SPIR-V 1.3 modules, above.

SPIR-V 1.4 does not add any new required capabilities.

3.6. SPIR-V 1.5

An OpenCL environment supporting SPIR-V 1.5 must support SPIR-V 1.5 modules that declare the capabilities required for SPIR-V 1.4 modules, above.

SPIR-V 1.5 does not add any new required capabilities.

3.7. SPIR-V 1.6

An OpenCL environment supporting SPIR-V 1.6 must support SPIR-V 1.6 modules that declare the capabilities required for SPIR-V 1.5 modules, above.

Prior to OpenCL 3.1, SPIR-V 1.6 did not add any new required capabilities.

An OpenCL environment supporting SPIR-V 1.6 and OpenCL 3.1 must support SPIR-V 1.6 modules that declare the following capabilities:

- **DotProduct**
- **DotProductInput4x8Bit** if `CL_DEVICE_INTEGER_DOT_PRODUCT_INPUT_4x8BIT` is supported
- **DotProductInput4x8BitPacked**

3.8. SPIR-V Extensions

An OpenCL environment supporting OpenCL 3.1 must support SPIR-V modules that declare the following SPIR-V extensions via **OpExtension** and the following extension capabilities:

- `SPV_KHR_bit_instructions`
 - **BitInstructions**
- `SPV_KHR_integer_dot_product`
 - **DotProductKHR**
 - **DotProductInput4x8BitKHR** if `CL_DEVICE_INTEGER_DOT_PRODUCT_INPUT_4x8BIT_KHR` is

supported

- **DotProductInput4x8BitPackedKHR**
- **SPV_KHR_subgroup_rotate**
 - **GroupNonUniformRotateKHR**

Chapter 4. Validation Rules

The following are a list of validation rules that apply to SPIR-V modules executing in all OpenCL environments:

The *Execution Model* declared in **OpEntryPoint** must be **Kernel**.

The *Addressing Model* declared in **OpMemoryModel** must be either:

- **Physical32** (for OpenCL devices reporting 32 for **CL_DEVICE_ADDRESS_BITS**)
- **Physical64** (for OpenCL devices reporting 64 for **CL_DEVICE_ADDRESS_BITS**)

The *Memory Model* declared in **OpMemoryModel** must be **OpenCL**.

For all **OpTypeInt** integer type-declaration instructions:

- *Signedness* must be 0, indicating no signedness semantics.

For all **OpTypeImage** type-declaration instructions:

- *Sampled Type* must be **OpTypeVoid**.
- *Sampled* must be 0, indicating that the image usage will be known at run time, not at compile time.
- *MS* must be 0, indicating single-sampled content.
- *Arrayed* may only be set to 1, indicating arrayed content, when *Dim* is set to **1D** or **2D**.
- *Image Format* must be **Unknown**, indicating that the image does not have a specified format.
- The optional image *Access Qualifier* must be present.

For all image read and write instructions, if an optional *Image Operand* is present, then it must not include **ConstOffset**.

For all **Atomic Instructions**:

- Only 32-bit integer types are supported for the *Result Type* and/or type of *Value*.
- The *Pointer* operand must be a pointer to the **Function**, **Workgroup**, or **CrossWorkgroup Storage Classes**. Note that an **Atomic Instruction** on a pointer to the **Function Storage Class** is valid, but does not have defined behavior.
- For OpenCL environments that support and declare the **GenericPointer** capability, the *Pointer* operand may be a pointer to the **Generic Storage Class**, however behavior is still undefined if the **Generic** pointer represents a pointer to the **Function Storage Class**.

Recursion is not supported. The static function call graph for an entry point must not contain cycles.

Whether irreducible control flow is legal is implementation-defined.

For the instructions **OpGroupAsyncCopy** and **OpGroupWaitEvents**, *Scope for Execution* must be:

- **Workgroup**

For the **Group and Subgroup Instructions**, *Scope for Execution* must be one of:

- **Workgroup**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Work-group Collective Functions (where `CL_DEVICE_WORK_GROUP_COLLECTIVE_FUNCTIONS_SUPPORT` is `CL_TRUE`).

- **Subgroup**

- For OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not 0)

For all other instructions, *Scope for Execution* must be one of:

- **Workgroup**

- **Subgroup**

- For OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not 0)

In an OpenCL 1.2 environment, for the **Barrier Instructions** **OpControlBarrier** and **OpMemoryBarrier**, the *Scope for Memory* must be **Workgroup**, and the memory-order constraint in *Memory Semantics* must be **SequentiallyConsistent**. Otherwise, *Scope for Memory* must be one of:

- **CrossDevice**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_ALL_DEVICES` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

- **Device**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_DEVICE` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

- **Workgroup**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_WORK_GROUP` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

- **Subgroup**

- For OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not 0).

- **Invocation**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_WORK_ITEM` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

And, the memory-order constraint in *Memory Semantics* must be one of:

- **None (Relaxed)**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_RELAXED` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

- **Acquire, Release, or AcquireRelease**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_ACQ_REL` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

- **SequentiallyConsistent**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_SEQ_CST` in `CL_DEVICE_ATOMIC_FENCE_CAPABILITIES`.

In all OpenCL environments, for the **Barrier Instruction** `OpControlBarrier`, when the *Scope* for *Execution* is **Subgroup**, behavior is undefined unless all invocations in the sub-group execute the same dynamic instance of the instruction.

In an OpenCL 1.2 environment, for the **Atomic Instructions**, the *Scope* for *Memory* must be **Device**, and the memory-order constraint in *Memory Semantics* must be **Relaxed**. Otherwise, *Scope* for *Memory* must be one of:

- **CrossDevice**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_ALL_DEVICES` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

- **Device**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_DEVICE` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

- **Workgroup**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_SCOPE_WORK_GROUP` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

- **Subgroup**

- For OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting Sub-groups (where `CL_DEVICE_MAX_NUM_SUB_GROUPS` is not 0).

And, the memory-order constraint in *Memory Semantics* must be one of:

- **None (Relaxed)**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_RELAXED` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

- **Acquire, Release, or AcquireRelease**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_ACQ_REL` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

- **SequentiallyConsistent**

- For OpenCL 2.0, OpenCL 2.1, OpenCL 2.2, or OpenCL 3.0 devices supporting `CL_DEVICE_ATOMIC_ORDER_SEQ_CST` in `CL_DEVICE_ATOMIC_MEMORY_CAPABILITIES`.

Chapter 5. OpenCL Extensions

An OpenCL environment may be modified by [OpenCL extensions](#). For example, some OpenCL extensions may require support for additional SPIR-V capabilities or instructions, or relax SPIR-V restrictions. Some OpenCL extensions may modify the OpenCL environment by requiring consumption of a SPIR-V module that uses a SPIR-V extension. In this case, the implementation will include the OpenCL extension in the host API `CL_PLATFORM_EXTENSIONS` or `CL_DEVICE_EXTENSIONS` string, but not the corresponding SPIR-V extension.

This section describes how the OpenCL environment is modified by Khronos (`KHR`) OpenCL extensions. Other OpenCL extensions, such as multi-vendor (`EXT`) extensions or vendor-specific extensions, describe how they modify the OpenCL environment in their individual extension specifications.

5.1. Declaring SPIR-V Extensions

A SPIR-V module declares use of a SPIR-V extension using **OpExtension** and the name of the SPIR-V extension. For example:

```
OpExtension "SPV_KHR_extension_name"
```

Only use of SPIR-V extensions may be declared in a SPIR-V module using **OpExtension**; there is never a need to declare use of an OpenCL extension in a SPIR-V module using **OpExtension**.

5.2. Full and Embedded Profile Extensions

5.2.1. `cl_khr_3d_image_writes`

If the OpenCL environment supports the extension `cl_khr_3d_image_writes`, then the environment must accept *Image* operands to **OpImageWrite** that are declared with dimensionality *Dim* equal to **3D**.

5.2.2. `cl_khr_depth_images`

If the OpenCL environment supports the extension `cl_khr_depth_images`, then the environment must accept modules that declare 2D depth image types using **OpTypeImage** with dimensionality *Dim* equal to **2D** and *Depth* equal to 1, indicating a depth image. 2D depth images may optionally be *Arrayed*, if supported.

Additionally, the following Image Channel Orders may be returned by **OpImageQueryOrder**:

- **Depth**

5.2.3. `cl_khr_device_enqueue_local_arg_types`

If the OpenCL environment supports the extension `cl_khr_device_enqueue_local_arg_types`, then the

environment will allow *Invoke* functions to be passed to **OpEnqueueKernel** with **Workgroup** memory pointer parameters of any type.

5.2.4. **cl_khr_fp16**

If the OpenCL environment supports the extension **cl_khr_fp16**, then the environment must accept modules that declare the following SPIR-V capabilities:

- **Float16**

5.2.5. **cl_khr_fp64**

If the OpenCL environment supports the extension **cl_khr_fp64**, then the environment must accept modules that declare the following SPIR-V capabilities:

- **Float64**

5.2.6. **cl_khr_gl_depth_images**

If the OpenCL environment supports the extension **cl_khr_gl_depth_images**, then the following Image Channel Orders may additionally be returned by **OpImageQueryOrder**:

- **DepthStencil**

Also, the following Image Channel Data Types may additionally be returned by **OpImageQueryFormat**:

- **UnormInt24**

5.2.7. **cl_khr_gl_msaa_sharing**

If the OpenCL environment supports the extension **cl_khr_gl_msaa_sharing**, then the environment must accept modules that declare 2D multi-sampled image types using **OpTypeImage** with dimensionality *Dim* equal to **2D** and *MS* equal to 1, indicating multi-sampled content. 2D multi-sampled images may optionally be *Arrayed* or *Depth* images, if supported.

The 2D multi-sampled images may be used with the following instructions:

- **OpImageRead**
- **OpImageQuerySizeLod**
- **OpImageQueryFormat**
- **OpImageQueryOrder**
- **OpImageQuerySamples**

5.2.8. **cl_khr_int64_base_atomics** and **cl_khr_int64_extended_atomics**

If the OpenCL environment supports the extension **cl_khr_int64_base_atomics** or **cl_khr_int64_extended_atomics**, then the environment must accept modules that declare the following SPIR-V

capabilities:

- **Int64Atomics**

When the **Int64Atomics** capability is declared, 64-bit integer types are valid for the *Result Type* and type of *Value* for all **Atomic Instructions**.

Note: OpenCL environments that consume SPIR-V must support both `cl_khr_int64_base_atomics` and `cl_khr_int64_extended_atomics` or neither of these extensions.

5.2.9. `cl_khr_mipmap_image`

If the OpenCL environment supports the extension `cl_khr_mipmap_image`, then the environment must accept non-zero optional **Lod Image Operands** for the following instructions:

- **OpImageSampleExplicitLod**
- **OpImageRead**
- **OpImageQuerySizeLod**

Note: Implementations that support `cl_khr_mipmap_image` are not guaranteed to support the **ImageMipmap** capability, since this extension does not require non-zero optional **Lod Image Operands** for **OpImageWrite**.

5.2.10. `cl_khr_mipmap_image_writes`

If the OpenCL environment supports the extension `cl_khr_mipmap_image_writes`, then the environment must accept non-zero optional **Lod Image Operands** for the following instructions:

- **OpImageWrite**

Note: An implementation that supports `cl_khr_mipmap_image_writes` must also support `cl_khr_mipmap_image`, and support for both extensions does guarantee support for the **ImageMipmap** capability.

5.2.11. `cl_khr_subgroups`

If the OpenCL environment supports the extension `cl_khr_subgroups`, then for all instructions except **OpGroupAsyncCopy** and **OpGroupWaitEvents** the *Scope for Execution* may be:

- **Subgroup**

Additionally, for all instructions except **Atomic Instructions** in an OpenCL 1.2 environment, the *Scope for Memory* may be:

- **Subgroup**

5.2.12. `cl_khr_subgroup_named_barrier`

If the OpenCL environment supports the extension `cl_khr_subgroup_named_barrier`, then the environment must accept modules that declare the following SPIR-V capabilities:

- **NamedBarrier**

5.2.13. **cl_khr_spirv_no_integer_wrap_decoration**

If the OpenCL environment supports the extension **cl_khr_spirv_no_integer_wrap_decoration**, then the environment must accept modules that declare use of the extension **SPV_KHR_no_integer_wrap_decoration** via **OpExtension**.

If the OpenCL environment supports the extension **cl_khr_spirv_no_integer_wrap_decoration** and use of the SPIR-V extension **SPV_KHR_no_integer_wrap_decoration** is declared in the module via **OpExtension**, then the environment must accept modules that include the **NoSignedWrap** or **NoUnsignedWrap** decorations.

5.2.14. **cl_khr_subgroup_extended_types**

If the OpenCL environment supports the extension **cl_khr_subgroup_extended_types**, then additional types are valid for the following for **Groups** instructions with *Scope* for *Execution* equal to **Subgroup**:

- **OpGroupBroadcast**
- **OpGroupIAdd, OpGroupFAdd**
- **OpGroupSMin, OpGroupUMin, OpGroupFMin**
- **OpGroupSMax, OpGroupUMax, OpGroupFMax**

For these instructions, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to **char**, **uchar**, **short**, **ushort**, **int**, **uint**, **long**, and **ulong**)
 - **OpTypeFloat** (equivalent to **half**, **float**, and **double**)

Additionally, for **OpGroupBroadcast**, valid types for *Value* are:

- **OpTypeVectors** with 2, 3, 4, 8, or 16 *Component Count* components of supported types:
 - **OpTypeInt** (equivalent to **char_n**, **uchar_n**, **short_n**, **ushort_n**, **int_n**, **uint_n**, **long_n**, and **ulong_n**)
 - **OpTypeFloat** (equivalent to **half_n**, **float_n**, and **double_n**)

5.2.15. **cl_khr_subgroup_non_uniform_vote**

If the OpenCL environment supports the extension **cl_khr_subgroup_non_uniform_vote**, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniform**
- **GroupNonUniformVote**

For instructions requiring these capabilities, *Scope* for *Execution* may be:

- **Subgroup**

For the instruction **OpGroupNonUniformAllEqual**, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to **char**, **uchar**, **short**, **ushort**, **int**, **uint**, **long**, and **ulong**)
 - **OpTypeFloat** (equivalent to **half**, **float**, and **double**)

5.2.16. **cl_khr_subgroup_ballot**

If the OpenCL environment supports the extension **cl_khr_subgroup_ballot**, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniformBallot**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

For the non-uniform broadcast instruction **OpGroupNonUniformBroadcast**, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to **char**, **uchar**, **short**, **ushort**, **int**, **uint**, **long**, and **ulong**)
 - **OpTypeFloat** (equivalent to **half**, **float**, and **double**)
- **OpTypeVectors** with 2, 3, 4, 8, or 16 *Component Count* components of supported types:
 - **OpTypeInt** (equivalent to **char_n**, **uchar_n**, **short_n**, **ushort_n**, **int_n**, **uint_n**, **long_n**, and **ulong_n**)
 - **OpTypeFloat** (equivalent to **half_n**, **float_n**, and **double_n**)

For the instruction **OpGroupNonUniformBroadcastFirst**, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to **char**, **uchar**, **short**, **ushort**, **int**, **uint**, **long**, and **ulong**)
 - **OpTypeFloat** (equivalent to **half**, **float**, and **double**)

For the instruction **OpGroupNonUniformBallot**, the valid *Result Type* is an **OpTypeVector** with four *Component Count* components of **OpTypeInt**, with *Width* equal to 32 and *Signedness* equal to 0 (equivalent to **uint4**).

For the instructions **OpGroupNonUniformInverseBallot**, **OpGroupNonUniformBallotBitExtract**, **OpGroupNonUniformBallotBitCount**, **OpGroupNonUniformBallotFindLSB**, and **OpGroupNonUniformBallotFindMSB**, the valid type for *Value* is an **OpTypeVector** with four *Component Count* components of **OpTypeInt**, with *Width* equal to 32 and *Signedness* equal to 0 (equivalent to **uint4**).

For built-in variables decorated with **SubgroupEqMask**, **SubgroupGeMask**, **SubgroupGtMask**, **SubgroupLeMask**, or **SubgroupLtMask**, the supported variable type is an **OpTypeVector** with four *Component Count* components of **OpTypeInt**, with *Width* equal to 32 and *Signedness* equal to 0 (equivalent to **uint4**).

5.2.17. `cl_khr_subgroup_non_uniform_arithmetic`

If the OpenCL environment supports the extension `cl_khr_subgroup_non_uniform_arithmetic`, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniformArithmetic**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

For the instructions **OpGroupNonUniformLogicalAnd**, **OpGroupNonUniformLogicalOr**, and **OpGroupNonUniformLogicalXor**, the valid type for *Value* is **OpTypeBool**.

Otherwise, for the **GroupNonUniformArithmetic** scan and reduction instructions, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to `char`, `uchar`, `short`, `ushort`, `int`, `uint`, `long`, and `ulong`)
 - **OpTypeFloat** (equivalent to `half`, `float`, and `double`)

For the **GroupNonUniformArithmetic** scan and reduction instructions, the optional *ClusterSize* operand must not be present.

5.2.18. `cl_khr_subgroup_shuffle`

If the OpenCL environment supports the extension `cl_khr_subgroup_shuffle`, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniformShuffle**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

For the instructions **OpGroupNonUniformShuffle** and **OpGroupNonUniformShuffleXor** requiring these capabilities, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to `char`, `uchar`, `short`, `ushort`, `int`, `uint`, `long`, and `ulong`)
 - **OpTypeFloat** (equivalent to `half`, `float`, and `double`)

5.2.19. `cl_khr_subgroup_shuffle_relative`

If the OpenCL environment supports the extension `cl_khr_subgroup_shuffle_relative`, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniformShuffleRelative**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

For the **GroupNonUniformShuffleRelative** instructions, valid types for *Value* are:

- Scalars of supported types:
 - **OpTypeInt** (equivalent to `char`, `uchar`, `short`, `ushort`, `int`, `uint`, `long`, and `ulong`)
 - **OpTypeFloat** (equivalent to `half`, `float`, and `double`)

5.2.20. `cl_khr_subgroup_clustered_reduce`

If the OpenCL environment supports the extension `cl_khr_subgroup_clustered_reduce`, then the environment must accept SPIR-V modules that declare the following SPIR-V capabilities:

- **GroupNonUniformClustered**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

When the **GroupNonUniformClustered** capability is declared, the **GroupNonUniformArithmetic** scan and reduction instructions may include the optional *ClusterSize* operand.

5.2.21. `cl_khr_spirv_extended_debug_info`

If the OpenCL environment supports the extension `cl_khr_spirv_extended_debug_info`, then the environment must accept modules that import the `OpenCL.DebugInfo.100` extended instruction set via **OpExtInstImport**.

5.2.22. `cl_khr_spirv_linkonce_odr`

If the OpenCL environment supports the extension `cl_khr_spirv_linkonce_odr`, then the environment must accept modules that declare use of the extension `SPV_KHR_linkonce_odr` via **OpExtension**.

If the OpenCL environment supports the extension `cl_khr_spirv_linkonce_odr` and use of the SPIR-V extension `SPV_KHR_linkonce_odr` is declared in the module via **OpExtension**, then the environment must accept modules that include the **LinkOnceODR** linkage type.

5.2.23. `cl_khr_extended_bit_ops`

If the OpenCL environment supports the extension `cl_khr_extended_bit_ops`, then the environment must accept modules that declare use of the extension `SPV_KHR_bit_instructions` via **OpExtension**.

If the OpenCL environment supports the extension `cl_khr_extended_bit_ops` and use of the SPIR-V extension `SPV_KHR_bit_instructions` is declared in the module via **OpExtension**, then the environment must accept modules that declare the **BitInstructions** capability.

5.2.24. `cl_khr_integer_dot_product`

If the OpenCL environment supports the extension `cl_khr_integer_dot_product`, then the environment must accept modules that require `SPV_KHR_integer_dot_product` and declare the following SPIR-V capabilities:

- **DotProductKHR**
- **DotProductInput4x8BitKHR** if `CL_DEVICE_INTEGER_DOT_PRODUCT_INPUT_4x8BIT_KHR` is supported
- **DotProductInput4x8BitPackedKHR**

If the OpenCL environment supports the extension `cl_khr_integer_dot_product` and SPIR-V 1.6, then the environment must accept SPIR-V 1.6 modules that declare the following SPIR-V capabilities:

- **DotProduct**
- **DotProductInput4x8Bit** if `CL_DEVICE_INTEGER_DOT_PRODUCT_INPUT_4x8BIT_KHR` is supported
- **DotProductInput4x8BitPacked**

5.2.25. `cl_khr_expect_assume`

If the OpenCL environment supports the extension `cl_khr_expect_assume`, then the environment must accept modules that declare use of the extension `SPV_KHR_expect_assume` via **OpExtension**.

If the OpenCL environment supports the extension `cl_khr_expect_assume` and use of the SPIR-V extension `SPV_KHR_expect_assume` is declared in the module via **OpExtension**, then the environment must accept modules that declare the following SPIR-V capabilities:

- **ExpectAssumeKHR**

5.2.26. `cl_khr_subgroup_rotate`

If the OpenCL environment supports the extension `cl_khr_subgroup_rotate`, then the environment accept modules that require `SPV_KHR_subgroup_rotate` and declare the following SPIR-V capabilities:

- **GroupNonUniformRotateKHR**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Subgroup**

5.2.27. `cl_khr_work_group_uniform_arithmetic`

If the OpenCL environment supports the extension `cl_khr_work_group_uniform_arithmetic`, then the environment must accept modules that declare use of the extension `SPV_KHR_uniform_group_instructions` via **OpExtension**.

If the OpenCL environment supports the extension `cl_khr_work_group_uniform_arithmetic` and use of the SPIR-V extension `SPV_KHR_uniform_group_instructions` is declared in the module via **OpExtension**, then the environment must accept modules that declare the following SPIR-V capabilities:

- **GroupUniformArithmeticKHR**

For instructions requiring these capabilities, *Scope for Execution* may be:

- **Workgroup**

For the instructions **OpGroupLogicalAndKHR**, **OpGroupLogicalOrKHR**, and **OpGroupLogicalXorKHR**, the valid type for *X* is **OpTypeBool**.

Otherwise, for the **GroupUniformArithmeticKHR** scan and reduction instructions, valid types for *X* are:

- Scalars of supported types:
 - **OpTypeInt** with *Width* equal to 32 or 64 (equivalent to **int**, **uint**, **long**, and **ulong**)
 - **OpTypeFloat** (equivalent to **half**, **float**, and **double**)

5.2.28. **cl_khr_kernel_clock**

If the OpenCL environment supports the extension **cl_khr_kernel_clock**, then the environment must accept modules that declare use of the extension **SPV_KHR_shader_clock** via **OpExtension**.

If the OpenCL environment supports the extension **cl_khr_kernel_clock** and use of the SPIR-V extension **SPV_KHR_shader_clock** is declared in the module via **OpExtension**, then the environment must accept modules that declare the following SPIR-V capability:

- **ShaderClockKHR**

For the **OpReadClockKHR** instruction requiring this capability, supported values for *Scope* are:

- **Device**, if **CL_DEVICE_KERNEL_CLOCK_SCOPE_DEVICE_KHR** is supported
- **Workgroup**, if **CL_DEVICE_KERNEL_CLOCK_SCOPE_WORK_GROUP_KHR** is supported
- **Subgroup**, if **CL_DEVICE_KERNEL_CLOCK_SCOPE_SUB_GROUP_KHR** is supported

For unsupported *Scope* values, the behavior of **OpReadClockKHR** is undefined.

5.2.29. **cl_khr_spirv_queries**

If the OpenCL environment supports the extension **cl_khr_spirv_queries**, then:

- For each of the strings returned by the query **CL_DEVICE_SPIRV_EXTENDED_INSTRUCTION_SETS_KHR**, it is valid to import the SPIR-V extended instruction set in a SPIR-V module using **OpExtInstImport**.
- For each of the strings returned by the query **CL_DEVICE_SPIRV_EXTENSIONS_KHR**, it is valid to declare use of the SPIR-V extension in a SPIR-V module using **OpExtension**.
- For each of the capabilities returned by the query **CL_DEVICE_SPIRV_CAPABILITIES_KHR**, it is valid to declare the SPIR-V capability in a SPIR-V module using **OpCapability**. Some capabilities may additionally require a specific SPIR-V version, or a SPIR-V extension to be declared via **OpExtension**.

5.3. Embedded Profile Extensions

5.3.1. `cles_khr_int64`

If the OpenCL environment supports the extension `cles_khr_int64`, then the environment must accept modules that declare the following SPIR-V capabilities:

- **Int64**

Chapter 6. OpenCL Numerical Compliance

This section describes features of the [C++14](#) and [IEEE-754](#) standards that must be supported by all OpenCL compliant devices.

This section describes the functionality that must be supported by all OpenCL devices for single precision floating-point numbers. Currently, only single precision floating-point is a requirement. Half precision floating-point is an optional feature indicated by the **Float16** capability. Double precision floating-point is also an optional feature indicated by the **Float64** capability.

6.1. Rounding Modes

Floating-point calculations may be carried out internally with extra precision and then rounded to fit into the destination type. IEEE 754 defines four possible rounding modes:

- *Round to nearest even*
- *Round toward +infinity*
- *Round toward -infinity*
- *Round toward zero*

The complete set of rounding modes supported by the device are described by the `CL_DEVICE_SINGLE_FP_CONFIG`, `CL_DEVICE_HALF_FP_CONFIG`, and `CL_DEVICE_DOUBLE_FP_CONFIG` device queries.

For double precision operations, *Round to nearest even* is a required rounding mode, and is therefore the default rounding mode for double precision operations.

For single precision operations, devices supporting the full profile must support *Round to nearest even*, therefore for full profile devices this is the default rounding mode for single precision operations. Devices supporting the embedded profile may support either *Round to nearest even* or *Round toward zero* as the default rounding mode for single precision operations.

For half precision operations, devices may support either *Round to nearest even* or *Round toward zero* as the default rounding mode for half precision operations.

Only static selection of rounding mode is supported. Dynamically reconfiguring the rounding mode as specified by the IEEE 754 spec is not supported.

6.2. Rounding Modes for Conversions

Results of the following conversion instructions may include an optional **FP Rounding Mode** decoration:

- `OpConvertFToU`
- `OpConvertFToS`
- `OpConvertSToF`
- `OpConvertUToF`

- **OpFConvert**

The **FPRoundingMode** decoration may not be added to results of any other instruction.

If no rounding mode is specified explicitly via an **FPRoundingMode** decoration, then the default rounding mode for conversion operations is:

- *Round to nearest even*, for conversions to floating-point types.
- *Round toward zero*, for conversions from floating-point to integer types.

6.3. Out-of-Range Conversions

When a conversion operand is either greater than the greatest representable destination value or less than the least representable destination value, it is said to be out-of-range.

Converting an out-of-range integer to an integer type without a **SaturatedConversion** decoration follows C99/C++14 conversion rules.

Converting an out-of-range floating-point number to an integer type without a **SaturatedConversion** decoration is implementation-defined.

6.4. INF, NaN, and Denormalized Numbers

INFs and NaNs must be supported. Support for signaling NaNs is not required.

Support for denormalized numbers with single precision and half precision floating-point is optional. Denormalized single precision or half precision floating-point numbers passed as the input or produced as the output of single precision or half precision floating-point operations may be flushed to zero. Support for denormalized numbers is required for double precision floating-point.

Support for INFs, NaNs, and denormalized numbers is described by the **CL_FP_DENORM** and **CL_FP_INF_NAN** bits in the **CL_DEVICE_SINGLE_FP_CONFIG**, **CL_DEVICE_HALF_FP_CONFIG**, and **CL_DEVICE_DOUBLE_FP_CONFIG** device queries.

6.5. Floating-Point Exceptions

Floating-point exceptions are disabled in OpenCL. The result of a floating-point exception must match the IEEE 754 spec for the exceptions-not-enabled case. Whether and when the implementation sets floating-point flags or raises floating-point exceptions is implementation-defined.

This standard provides no method for querying, clearing or setting floating-point flags or trapping raised exceptions. Due to non-performance, non-portability of trap mechanisms, and the impracticality of servicing precise exceptions in a vector context (especially on heterogeneous hardware), such features are discouraged.

Implementations that nevertheless support such operations through an extension to the standard

shall initialize with all exception flags cleared and the exception masks set so that exceptions raised by arithmetic operations do not trigger a trap to be taken. If the underlying work is reused by the implementation, the implementation is however not responsible for re-clearing the flags or resetting exception masks to default values before entering the kernel. That is to say that kernels that do not inspect flags or enable traps are licensed to expect that their arithmetic will not trigger a trap. Those kernels that do examine flags or enable traps are responsible for clearing flag state and disabling all traps before returning control to the implementation. Whether or when the underlying work-item (and accompanying global floating-point state if any) is reused is implementation-defined.

6.6. Relative Error as ULPs

In this section we discuss the maximum relative error defined as ulp (units in the last place). Addition, subtraction, multiplication, fused multiply-add, and conversion between integer and a single precision floating-point format are IEEE 754 compliant and are therefore correctly rounded. Conversion between floating-point formats and explicit conversions must be correctly rounded.

The ULP is defined as follows:

If x is a real number that lies between two finite consecutive floating-point numbers a and b , without being equal to one of them, then $\text{ulp}(x) = |b - a|$, otherwise $\text{ulp}(x)$ is the distance between the two non-equal finite floating-point numbers nearest x . Moreover, $\text{ulp}(\text{NaN})$ is NaN.

Attribution: This definition was taken with consent from Jean-Michel Muller with slight clarification for behavior at zero. Refer to: [On the definition of ulp\(x\)](#).

0 ULP is used for math functions that do not require rounding. The reference value used to compute the ULP value is the infinitely precise result.

Result overflow within the specified ULP error is permitted. Math instructions are allowed to return infinity for a finite reference value when the next floating-point number that would be representable after the finite maximum, if there was sufficient range, meets ULP error tolerance.

6.6.1. ULP Values for Math Instructions - Full Profile

The ULP Values for Math Instructions - Full Profile table below describes the minimum accuracy of floating-point math arithmetic instructions given as ULP values for the full profile.

Table 5. ULP Values for Math Instructions - Full Profile

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpFAdd	Correctly rounded	Correctly rounded	Correctly rounded
OpFSub	Correctly rounded	Correctly rounded	Correctly rounded
OpFMul	Correctly rounded	Correctly rounded	Correctly rounded

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpFDiv	Correctly rounded	≤ 2.5 ulp	≤ 1 ulp
OpExtInst acos	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst acosh	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst acospi	≤ 5 ulp	≤ 5 ulp	≤ 2 ulp
OpExtInst asin	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst asinh	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst asinpi	≤ 5 ulp	≤ 5 ulp	≤ 2 ulp
OpExtInst atan	≤ 5 ulp	≤ 5 ulp	≤ 2 ulp
OpExtInst atanh	≤ 5 ulp	≤ 5 ulp	≤ 2 ulp
OpExtInst atanpi	≤ 5 ulp	≤ 5 ulp	≤ 2 ulp
OpExtInst atan2	≤ 6 ulp	≤ 6 ulp	≤ 2 ulp
OpExtInst atan2pi	≤ 6 ulp	≤ 6 ulp	≤ 2 ulp
OpExtInst cbrt	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst ceil	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst copysign	0 ulp	0 ulp	0 ulp
OpExtInst cos	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst cosh	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst cospi	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst cross	absolute error tolerance of ' $\max * \max * (3 * \text{FLT_EPSILON})$ ' per vector component, where <i>max</i> is the maximum input operand magnitude	absolute error tolerance of ' $\max * \max * (3 * \text{FLT_EPSILON})$ ' per vector component, where <i>max</i> is the maximum input operand magnitude	absolute error tolerance of ' $\max * \max * (3 * \text{HALF_EPSILON})$ ' per vector component, where <i>max</i> is the maximum input operand magnitude
OpExtInst degrees	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst distance	$\leq 5.5 + 2n$ ulp, for gentype with vector width <i>n</i>	$\leq 2.5 + 2n$ ulp, for gentype with vector width <i>n</i>	$\leq 2n$ ulp, for gentype with vector width <i>n</i>

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpDot	absolute error tolerance of 'max * max * (2n - 1) * FLT_EPSILON', for vector width n and maximum input operand magnitude max across all vector components	absolute error tolerance of 'max * max * (2n - 1) * FLT_EPSILON', for vector width n and maximum input operand magnitude max across all vector components	absolute error tolerance of 'max * max * (2n - 1) * HALF_EPSILON', for vector width n and maximum input operand magnitude max across all vector components
OpExtInst erfc	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst erf	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst exp	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst exp2	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst exp10	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst expm1	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst fabs	0 ulp	0 ulp	0 ulp
OpExtInst fclamp	0 ulp	0 ulp	0 ulp
OpExtInst fdim	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst floor	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst fma	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst fmax	0 ulp	0 ulp	0 ulp
OpExtInst fmax_common	0 ulp	0 ulp	0 ulp
OpExtInst fmin	0 ulp	0 ulp	0 ulp
OpExtInst fmin_common	0 ulp	0 ulp	0 ulp
OpExtInst fmod	0 ulp	0 ulp	0 ulp
OpExtInst fract	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst frexp	0 ulp	0 ulp	0 ulp
OpExtInst hypot	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst ilogb	0 ulp	0 ulp	0 ulp
OpExtInst ldexp	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst length	$\leq 5.5 + n$ ulp, for gentype with vector width n	$\leq 2.75 + 0.5n$ ulp, for gentype with vector width n	$\leq 0.25 + 0.5n$ ulp, for gentype with vector width n
OpExtInst lgamma	Implementation-defined	Implementation-defined	Implementation-defined

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst lgamma_r	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst log	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst log2	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst log10	≤ 3 ulp	≤ 3 ulp	≤ 2 ulp
OpExtInst log1p	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst logb	0 ulp	0 ulp	0 ulp
OpExtInst mad	Implemented either as a correctly rounded fma, or as a multiply followed by an add, both of which are correctly rounded	Implemented either as a correctly rounded fma, or as a multiply followed by an add, both of which are correctly rounded	Implemented either as a correctly rounded fma, or as a multiply followed by an add, both of which are correctly rounded
OpExtInst maxmag	0 ulp	0 ulp	0 ulp
OpExtInst minmag	0 ulp	0 ulp	0 ulp
OpExtInst mix	Implementation-defined	absolute error tolerance of 1e-3	Implementation-defined
OpExtInst modf	0 ulp	0 ulp	0 ulp
OpExtInst nan	0 ulp	0 ulp	0 ulp
OpExtInst nextafter	0 ulp	0 ulp	0 ulp
OpExtInst normalize	$\leq 4.5 + n$ ulp, for gentype with vector width n	$\leq 2 + n$ ulp, for gentype with vector width n	$\leq 1 + n$ ulp, for gentype with vector width n
OpExtInst pow	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst pown	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst powr	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst radians	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst remainder	0 ulp	0 ulp	0 ulp
OpExtInst remquo	0 ulp for the remainder, at least the lower 7 bits of the integral quotient	0 ulp for the remainder, at least the lower 7 bits of the integral quotient	0 ulp for the remainder, at least the lower 7 bits of the integral quotient
OpExtInst rint	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst rootn	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst round	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst rsqrt	≤ 2 ulp	≤ 2 ulp	≤ 1 ulp

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst sign	0 ulp	0 ulp	0 ulp
OpExtInst sin	<= 4 ulp	<= 4 ulp	<= 2 ulp
OpExtInst sincos	<= 4 ulp for sine and cosine values	<= 4 ulp for sine and cosine values	<= 2 ulp for sine and cosine values
OpExtInst sinh	<= 4 ulp	<= 4 ulp	<= 2 ulp
OpExtInst sinpi	<= 4 ulp	<= 4 ulp	<= 2 ulp
OpExtInst smoothstep	Implementation-defined	absolute error tolerance of 1e-5	Implementation-defined
OpExtInst sqrt	Correctly rounded	<= 3 ulp	<= 1.5 ulp
OpExtInst step	0 ulp	0 ulp	0 ulp
OpExtInst tan	<= 5 ulp	<= 5 ulp	<= 2.5 ulp
OpExtInst tanh	<= 5 ulp	<= 5 ulp	<= 2 ulp
OpExtInst tanpi	<= 6 ulp	<= 6 ulp	<= 2 ulp
OpExtInst tgamma	<= 16 ulp	<= 16 ulp	<= 4 ulp
OpExtInst trunc	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst half_cos		<= 8192 ulp	
OpExtInst half_divide		<= 8192 ulp	
OpExtInst half_exp		<= 8192 ulp	
OpExtInst half_exp2		<= 8192 ulp	
OpExtInst half_exp10		<= 8192 ulp	
OpExtInst half_log		<= 8192 ulp	
OpExtInst half_log2		<= 8192 ulp	
OpExtInst half_log10		<= 8192 ulp	
OpExtInst half_powr		<= 8192 ulp	
OpExtInst half_recip		<= 8192 ulp	
OpExtInst half_rsqrt		<= 8192 ulp	
OpExtInst half_sin		<= 8192 ulp	
OpExtInst half_sqrt		<= 8192 ulp	
OpExtInst half_tan		<= 8192 ulp	
OpExtInst fast_distance		<= 8191.5 + 2n ulp, for gentype with vector width n	

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst fast_length		$\leq 8191.5 + n \text{ ulp}$, for gentype with vector width n	
OpExtInst fast_normalize		$\leq 8192 + n \text{ ulp}$, for gentype with vector width n	
OpExtInst native_cos		Implementation-defined	
OpExtInst native_divide		Implementation-defined	
OpExtInst native_exp		Implementation-defined	
OpExtInst native_exp2		Implementation-defined	
OpExtInst native_exp10		Implementation-defined	
OpExtInst native_log		Implementation-defined	
OpExtInst native_log2		Implementation-defined	
OpExtInst native_log10		Implementation-defined	
OpExtInst native_powr		Implementation-defined	
OpExtInst native_recip		Implementation-defined	
OpExtInst native_rsqrt		Implementation-defined	
OpExtInst native_sin		Implementation-defined	
OpExtInst native_sqrt		Implementation-defined	
OpExtInst native_tan		Implementation-defined	

6.6.2. ULP Values for Math Instructions - Embedded Profile

The ULP Values for Math instructions - Embedded Profile table below describes the minimum accuracy of floating-point math arithmetic instructions given as ULP values for the embedded

profile.

Table 6. ULP Values for Math Instructions - Embedded Profile

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpFAdd	Correctly rounded	Correctly rounded	Correctly rounded
OpFSub	Correctly rounded	Correctly rounded	Correctly rounded
OpFMul	Correctly rounded	Correctly rounded	Correctly rounded
OpFDiv	≤ 3 ulp	≤ 3 ulp	≤ 1 ulp
OpExtInst acos	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst acosh	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst acospi	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst asin	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst asinh	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst asinpi	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst atan	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst atanh	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst atanpi	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst atan2	≤ 6 ulp	≤ 6 ulp	≤ 3 ulp
OpExtInst atan2pi	≤ 6 ulp	≤ 6 ulp	≤ 3 ulp
OpExtInst cbrt	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst ceil	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst copysign	0 ulp	0 ulp	0 ulp
OpExtInst cos	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst cosh	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst cospi	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst cross	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst degrees	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst distance	Implementation-defined	Implementation-defined	Implementation-defined
OpDot	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst erfc	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst erf	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp
OpExtInst exp	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst exp2	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst exp10	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst expm1	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst fabs	0 ulp	0 ulp	0 ulp
OpExtInst fclamp	0 ulp	0 ulp	0 ulp
OpExtInst fdim	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst floor	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst fma	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst fmax	0 ulp	0 ulp	0 ulp
OpExtInst fmax_common	0 ulp	0 ulp	0 ulp
OpExtInst fmin	0 ulp	0 ulp	0 ulp
OpExtInst fmin_common	0 ulp	0 ulp	0 ulp
OpExtInst fmod	0 ulp	0 ulp	0 ulp
OpExtInst fract	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst frexp	0 ulp	0 ulp	0 ulp
OpExtInst hypot	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst ilogb	0 ulp	0 ulp	0 ulp
OpExtInst ldexp	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst length	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst lgamma	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst lgamma_r	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst log	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst log2	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst log10	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst log1p	<= 4 ulp	<= 4 ulp	<= 3 ulp
OpExtInst logb	0 ulp	0 ulp	0 ulp
OpExtInst mad	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst maxmag	0 ulp	0 ulp	0 ulp
OpExtInst minmag	0 ulp	0 ulp	0 ulp

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst mix	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst modf	0 ulp	0 ulp	0 ulp
OpExtInst nan	0 ulp	0 ulp	0 ulp
OpExtInst nextafter	0 ulp	0 ulp	0 ulp
OpExtInst normalize	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst pow	≤ 16 ulp	≤ 16 ulp	≤ 5 ulp
OpExtInst pown	≤ 16 ulp	≤ 16 ulp	≤ 5 ulp
OpExtInst powr	≤ 16 ulp	≤ 16 ulp	≤ 5 ulp
OpExtInst radians	≤ 2 ulp	≤ 2 ulp	≤ 2 ulp
OpExtInst remainder	0 ulp	0 ulp	0 ulp
OpExtInst remquo	0 ulp for the remainder, at least the lower 7 bits of the integral quotient	0 ulp for the remainder, at least the lower 7 bits of the integral quotient	0 ulp for the remainder, at least the lower 7 bits of the integral quotient
OpExtInst rint	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst rootn	≤ 16 ulp	≤ 16 ulp	≤ 5 ulp
OpExtInst round	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst rsqrt	≤ 4 ulp	≤ 4 ulp	≤ 1 ulp
OpExtInst sign	0 ulp	0 ulp	0 ulp
OpExtInst sin	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst sincos	≤ 4 ulp for sine and cosine values	≤ 4 ulp for sine and cosine values	≤ 2 ulp for sine and cosine values
OpExtInst sinh	≤ 4 ulp	≤ 4 ulp	≤ 3 ulp
OpExtInst sinpi	≤ 4 ulp	≤ 4 ulp	≤ 2 ulp
OpExtInst smoothstep	Implementation-defined	Implementation-defined	Implementation-defined
OpExtInst sqrt	≤ 4 ulp	≤ 4 ulp	≤ 1.5 ulp
OpExtInst step	0 ulp	0 ulp	0 ulp
OpExtInst tan	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst tanh	≤ 5 ulp	≤ 5 ulp	≤ 3 ulp
OpExtInst tanpi	≤ 6 ulp	≤ 6 ulp	≤ 3 ulp
OpExtInst tgamma	≤ 16 ulp	≤ 16 ulp	≤ 4 ulp

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst trunc	Correctly rounded	Correctly rounded	Correctly rounded
OpExtInst half_cos		<= 8192 ulp	
OpExtInst half_divide		<= 8192 ulp	
OpExtInst half_exp		<= 8192 ulp	
OpExtInst half_exp2		<= 8192 ulp	
OpExtInst half_exp10		<= 8192 ulp	
OpExtInst half_log		<= 8192 ulp	
OpExtInst half_log2		<= 8192 ulp	
OpExtInst half_log10		<= 8192 ulp	
OpExtInst half_powr		<= 8192 ulp	
OpExtInst half_recip		<= 8192 ulp	
OpExtInst half_rsqrt		<= 8192 ulp	
OpExtInst half_sin		<= 8192 ulp	
OpExtInst half_sqrt		<= 8192 ulp	
OpExtInst half_tan		<= 8192 ulp	
OpExtInst fast_distance		Implementation-defined	
OpExtInst fast_length		Implementation-defined	
OpExtInst fast_normalize		Implementation-defined	
OpExtInst native_cos		Implementation-defined	
OpExtInst native_divide		Implementation-defined	
OpExtInst native_exp		Implementation-defined	
OpExtInst native_exp2		Implementation-defined	
OpExtInst native_exp10		Implementation-defined	
OpExtInst native_log		Implementation-defined	
OpExtInst native_log2		Implementation-defined	

SPIR-V Instruction	Minimum Accuracy - Float64	Minimum Accuracy - Float32	Minimum Accuracy - Float16
OpExtInst native_log10		Implementation-defined	
OpExtInst native_powr		Implementation-defined	
OpExtInst native_recip		Implementation-defined	
OpExtInst native_rsqrt		Implementation-defined	
OpExtInst native_sin		Implementation-defined	
OpExtInst native_sqrt		Implementation-defined	
OpExtInst native_tan		Implementation-defined	

6.6.3. ULP Values for Math Instructions - Unsafe Math Optimizations Enabled

The ULP Values for Math Instructions with Unsafe Math Optimizations table below describes the minimum accuracy of commonly used single precision floating-point math arithmetic instructions given as ULP values if the *-cl-unsafe-math-optimizations* compiler option is specified when compiling or building the OpenCL program.

For derived implementations, the operations used in the derivation may themselves be relaxed according to the ULP Values for Math Instructions with Unsafe Math Optimizations table.

The minimum accuracy of math functions not defined in the ULP Values for Math Instructions with Unsafe Math Optimizations table when the *-cl-unsafe-math-optimizations* compiler option is specified is as defined in the [ULP Values for Math Instructions for Full Profile](#) table when operating in the full profile, and as defined in the [ULP Values for Math instructions for Embedded Profile](#) table when operating in the embedded profile.

Table 7. ULP Values for Single Precision Math Instructions with *-cl-unsafe-math-optimizations*

Function	Minimum Accuracy
OpFDiv for $1.0 / x$	≤ 2.5 ulp for x in the domain of 2^{-126} to 2^{126} for the full profile, and ≤ 3 ulp for x in the domain of 2^{-126} to 2^{126} for the embedded profile.
OpFDiv for x / y	≤ 2.5 ulp for x in the domain of 2^{-62} to 2^{62} and y in the domain of 2^{-62} to 2^{62} for the full profile, and ≤ 3 ulp for x in the domain of 2^{-62} to 2^{62} and y in the domain of 2^{-62} to 2^{62} for the embedded profile.
OpExtInst acos	≤ 4096 ulp

Function	Minimum Accuracy
OpExtInst acosh	Derived implementations may implement as $\log(x + \sqrt{x * x - 1})$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst acospi	Derived implementations may implement as $\text{acos}(x) * M_PI_F$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst asin	≤ 4096 ulp
OpExtInst asinh	Derived implementations may implement as $\log(x + \sqrt{x * x + 1})$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst asinpi	Derived implementations may implement as $\text{asin}(x) * M_PI_F$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst atan	≤ 4096 ulp
OpExtInst atanh	Defined for x in the domain $(-1, 1)$. For x in $[-2^{-10}, 2^{-10}]$, derived implementations may implement as x . For x outside of $[-2^{-10}, 2^{-10}]$, derived implementations may implement as $0.5f * \log((1.0f + x) / (1.0f - x))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst atanpi	Derived implementations may implement as $\text{atan}(x) * M_1_PI_F$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst atan2	Derived implementations may implement as $\text{atan}(y / x)$ for $x > 0$, $\text{atan}(y / x) + M_PI_F$ for $x < 0$ and $y > 0$, and $\text{atan}(y / x) - M_PI_F$ for $x < 0$ and $y < 0$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst atan2pi	Derived implementations may implement as $\text{atan2}(y, x) * M_1_PI_F$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst cbrt	Derived implementations may implement as $\text{rootn}(x, 3)$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst cos	For x in the domain $[-\pi, \pi]$, the maximum absolute error is $\leq 2^{-11}$ and larger otherwise.
OpExtInst cosh	Defined for x in the domain $[-88, 88]$. Derived implementations may implement as $0.5f * (\exp(x) + \exp(-x))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst cospi	For x in the domain $[-1, 1]$, the maximum absolute error is $\leq 2^{-11}$ and larger otherwise.
OpExtInst exp	$\leq 3 + \text{floor}(\text{fabs}(2 * x))$ ulp for the full profile, and $\leq 4 + \text{floor}(\text{fabs}(2 * x))$ ulp for the embedded profile.
OpExtInst exp2	$\leq 3 + \text{floor}(\text{fabs}(2 * x))$ ulp for the full profile, and $\leq 4 + \text{floor}(\text{fabs}(2 * x))$ ulp for the embedded profile.
OpExtInst exp10	Derived implementations may implement as $\exp2(x * \log2(10))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst expm1	Derived implementations may implement as $\exp(x) - 1$. For non-derived implementations, the error is ≤ 8192 ulp.

Function	Minimum Accuracy
OpExtInst log	For x in the domain $[0.5, 2]$ the maximum absolute error is $\leq 2^{-21}$; otherwise the maximum error is ≤ 3 ulp for the full profile and ≤ 4 ulp for the embedded profile.
OpExtInst log2	For x in the domain $[0.5, 2]$ the maximum absolute error is $\leq 2^{-21}$; otherwise the maximum error is ≤ 3 ulp for the full profile and ≤ 4 ulp for the embedded profile.
OpExtInst log10	For x in the domain $[0.5, 2]$ the maximum absolute error is $\leq 2^{-21}$; otherwise the maximum error is ≤ 3 ulp for the full profile and ≤ 4 ulp for the embedded profile.
OpExtInst log1p	Derived implementations may implement as $\log(x + 1)$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst pow	Undefined for $x = 0$ and $y = 0$. Undefined for $x < 0$ and non-integer y. Undefined for $x < 0$ and y outside the domain $[-2^{24}, 2^{24}]$. For $x > 0$ or $x < 0$ and even y, derived implementations may implement as $\exp2(y * \log2(\text{fabs}(x)))$. For $x < 0$ and odd y, derived implementations may implement as $-\exp2(y * \log2(\text{fabs}(x)))$. For $x == 0$ and non-zero y, for derived implementations may return zero. For non-derived implementations, the error is ≤ 8192 ulp. On some implementations, <code>powr()</code> or <code>pown()</code> may perform faster than <code>pow()</code>. If x is known to be ≥ 0, consider using <code>powr()</code> in place of <code>pow()</code>, or if y is known to be an integer, consider using <code>pown()</code> in place of <code>pow()</code>.
OpExtInst pown	Defined only for integer values of y . Undefined for $x = 0$ and $y = 0$. For $x \geq 0$ or $x < 0$ and even y , derived implementations may implement as $\exp2(y * \log2(\text{fabs}(x)))$. For $x < 0$ and odd y , derived implementations may implement as $-\exp2(y * \log2(\text{fabs}(x)))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst powr	Defined only for $x \geq 0$. Undefined for $x = 0$ and $y = 0$. Derived implementations may implement as $\exp2(y * \log2(x))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst rootn	Defined for $x > 0$ when y is non-zero, derived implementations may implement this case as $\exp2(\log2(x) / y)$. Defined for $x < 0$ when y is odd, derived implementations may implement this case as $-\exp2(\log2(-x) / y)$. Defined for $x = +/-0$ when $y > 0$, derived implementations may return $+0$ in this case. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst sin	For x in the domain $[-\pi, \pi]$, the maximum absolute error is $\leq 2^{-11}$ and larger otherwise.
OpExtInst sincos	ulp values as defined for $\sin(x)$ and $\cos(x)$.

Function	Minimum Accuracy
OpExtInst sinh	Defined for x in the domain $[-88, 88]$. For x in $[-2^{-10}, 2^{-10}]$, derived implementations may implement as x . For x outside of $[-2^{-10}, 2^{-10}]$, derived implementations may implement as $0.5f * (\exp(x) - \exp(-x))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst sinpi	For x in the domain $[-1, 1]$, the maximum absolute error is $\leq 2^{-11}$ and larger otherwise.
OpExtInst tan	Derived implementations may implement as $\sin(x) * (1.0f / \cos(x))$. For non-derived implementations, the error is ≤ 8192 ulp.
OpExtInst tanh	Defined for x in the domain $[-\infty, \infty]$. For x in $[-2^{-10}, 2^{-10}]$, derived implementations may implement as x . For x outside of $[-2^{-10}, 2^{-10}]$, derived implementations may implement as $(\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$. For non-derived implementations, the error is ≤ 8192 ULP.
OpExtInst tanpi	Derived implementations may implement as $\tan(x * M_PI_F)$. For non-derived implementations, the error is ≤ 8192 ulp for x in the domain $[-1, 1]$.
OpFMul and OpFAdd , for $x * y + z$	Implemented either as a correctly rounded fma or as a multiply and an add both of which are correctly rounded.

6.7. Edge Case Behavior

The edge case behavior of the math functions shall conform to sections F.9 and G.6 of ISO/IEC 9899:TC2, except where noted below in the [Additional Requirements Beyond ISO/IEC 9899:TC2 section](#).

The usual allowances for rounding error ([Relative Error as ULPs section](#)) or flushing behavior ([Edge Case Behavior in Flush To Zero Mode section](#)) shall not apply for those values for which [section F.9](#) of ISO/IEC 9899:TC2, or [Additional Requirements Beyond ISO/IEC 9899:TC2](#) and [Edge Case Behavior in Flush To Zero Mode sections](#) below (and similar sections for other floating-point precisions) prescribe a result (e.g. $\text{ceil}(-1 < x < 0)$ returns -0). Those values shall produce exactly the prescribed answers, and no other. Where the $\pm$ symbol is used, the sign shall be preserved. For example, $\sin(\pm 0) = \pm 0$ shall be interpreted to mean $\sin(+0)$ is +0 and $\sin(-0)$ is -0.

Some of the required edge case behavior described in sections F.9 and G.6 of ISO/IEC 9899:TC2 is:

- **OpExtInst acos:**
 - $\text{acos}(1)$ returns +0.
 - $\text{acos}(x)$ returns a NaN for $|x| > 1$.
- **OpExtInst acosh:**
 - $\text{acosh}(1)$ returns +0.
 - $\text{acosh}(x)$ returns a NaN for $x < 1$.
 - $\text{acosh}(+\infty)$ returns $+\infty$.

- **OpExtInst asin:**
 - $\text{asin}(\pm 0)$ returns ± 0 .
 - $\text{asin}(x)$ returns a NaN for $|x| > 1$.
- **OpExtInst asinh:**
 - $\text{asinh}(\pm 0)$ returns ± 0 .
 - $\text{asinh}(\pm\infty)$ returns $\pm\infty$.
- **OpExtInst atan:**
 - $\text{atan}(\pm 0)$ returns ± 0 .
 - $\text{atan}(\pm\infty)$ returns $\pm\pi / 2$.
- **OpExtInst atanh:**
 - $\text{atanh}(\pm 0)$ returns ± 0 .
 - $\text{atanh}(\pm 1)$ returns $\pm\infty$.
 - $\text{atanh}(x)$ returns a NaN for $|x| > 1$.
- **OpExtInst atan2:**
 - $\text{atan2}(\pm 0, -0)$ returns $\pm\pi$.
 - $\text{atan2}(\pm 0, +0)$ returns ± 0 .
 - $\text{atan2}(\pm 0, x)$ returns $\pm\pi$ for $x < 0$.
 - $\text{atan2}(\pm 0, x)$ returns ± 0 for $x > 0$.
 - $\text{atan2}(y, \pm 0)$ returns $-\pi / 2$ for $y < 0$.
 - $\text{atan2}(y, \pm 0)$ returns $\pi / 2$ for $y > 0$.
 - $\text{atan2}(\pm y, -\infty)$ returns $\pm\pi$ for finite $y > 0$.
 - $\text{atan2}(\pm y, +\infty)$ returns ± 0 for finite $y > 0$.
 - $\text{atan2}(\pm\infty, x)$ returns $\pm\pi / 2$ for finite x .
 - $\text{atan2}(\pm\infty, -\infty)$ returns ± 3 times $\pi / 4$.
 - $\text{atan2}(\pm\infty, +\infty)$ returns $\pm\pi / 4$.
- **OpExtInst cbrt:**
 - $\text{cbrt}(\pm 0)$ returns ± 0 .
 - $\text{cbrt}(\pm\infty)$ returns $\pm\infty$.
- **OpExtInst ceil:**
 - $\text{ceil}(\pm 0)$ returns ± 0 .
 - $\text{ceil}(\pm\infty)$ returns $\pm\infty$.
- **OpExtInst cos:**
 - $\text{cos}(\pm 0)$ returns 1.
 - $\text{cos}(\pm\infty)$ returns a NaN.
- **OpExtInst cosh:**

- $\cosh(\pm 0)$ returns 1.
- $\cosh(\pm\infty)$ returns $+\infty$.
- **OpExtInst erfc:**
 - $\text{erfc}(-\infty)$ returns 2.
 - $\text{erfc}(+\infty)$ returns +0.
- **OpExtInst erf:**
 - $\text{erf}(\pm 0)$ returns ± 0 .
 - $\text{erf}(\pm\infty)$ returns ± 1 .
- **OpExtInst exp:**
 - $\exp(\pm 0)$ returns 1.
 - $\exp(-\infty)$ returns +0.
 - $\exp(+\infty)$ returns $+\infty$.
- **OpExtInst exp2:**
 - $\exp2(\pm 0)$ returns 1.
 - $\exp2(-\infty)$ returns +0.
 - $\exp2(+\infty)$ returns $+\infty$.
- **OpExtInst expm1:**
 - $\expm1(\pm 0)$ returns ± 0 .
 - $\expm1(-\infty)$ returns -1.
 - $\expm1(+\infty)$ returns $+\infty$.
- **OpExtInst fabs:**
 - $\text{fabs}(\pm 0)$ returns +0.
 - $\text{fabs}(\pm\infty)$ returns $+\infty$.
- **OpExtInst floor:**
 - $\text{floor}(\pm 0)$ returns ± 0 .
 - $\text{floor}(\pm\infty)$ returns $\pm\infty$.
- **OpExtInst fma:**
 - $\text{fma}(x, y, z)$ returns a NaN if one of x and y is infinite and the other is zero.
 - $\text{fma}(x, y, z)$ returns a NaN if x times y is an exact infinity and z is also an infinity but with the opposite sign.
- **OpExtInst fmod:**
 - $\text{fmod}(\pm 0, y)$ returns ± 0 for y not zero.
 - $\text{fmod}(x, y)$ returns a NaN for x infinite or y zero.
 - $\text{fmod}(x, \pm\infty)$ returns x for x not infinite.
- **OpExtInst frexp:**

- `frexp( ±0, exp )` returns ± 0 , and stores 0 in the object pointed to by `exp`.

- **OpExtInst hypot:**

- `hypot( x, y )`, `hypot( y, x )`, and `hypot( x, -y )` are equivalent.
- `hypot( x, ±0 )` is equivalent to `fabs( x )`.
- `hypot( ±∞, y )` returns $+\infty$, even if `y` is a NaN.

- **OpExtInst lgamma:**

- `lgamma( 1 )` returns $+0$.
- `lgamma( 2 )` returns $+0$.
- `lgamma( x )` returns $+\infty$ for `x` a negative integer or zero.
- `lgamma( -∞ )` returns $+\infty$.
- `lgamma( +∞ )` returns $+\infty$.

- **OpExtInst log:**

- `log( ±0 )` returns $-\infty$.
- `log( 1 )` returns $+0$.
- `log( x )` returns a NaN for `x` < 0 .
- `log( +∞ )` returns $+\infty$.

- **OpExtInst log2:**

- `log2( ±0 )` returns $-\infty$.
- `log2( 1 )` returns $+0$.
- `log2( x )` returns a NaN for `x` < 0 .
- `log2( +∞ )` returns $+\infty$.

- **OpExtInst log10:**

- `log10( ±0 )` returns $-\infty$.
- `log10( 1 )` returns $+0$.
- `log10( x )` returns a NaN for `x` < 0 .
- `log10( +∞ )` returns $+\infty$.

- **OpExtInst log1p:**

- `log1p( ±0 )` returns ± 0 .
- `log1p( -1 )` returns $-\infty$.
- `log1p( x )` returns a NaN for `x` < -1 .
- `log1p( +∞ )` returns $+\infty$.

- **OpExtInst logb:**

- `logb( ±0 )` returns $-\infty$.
- `logb( ±∞ )` returns $+\infty$.

- **OpExtInst modf:**

- `modf( ±x, iptr )` returns a result with the same sign as x .
- `modf( ±∞, iptr )` returns $±0$ and stores $±∞$ in the object pointed to by `iptr`.
- `modf( NaN, iptr )` stores a NaN in the object pointed to by `iptr` (and returns a NaN).

• **OpExtInst pow:**

- `pow( ±0, y )` returns $±∞$ for y an odd integer < 0 .
- `pow( ±0, y )` returns $+∞$ for $y < 0$ and not an odd integer.
- `pow( ±0, y )` returns $±0$ for y an odd integer > 0 .
- `pow( ±0, y )` returns $+0$ for $y > 0$ and not an odd integer.
- `pow( -1, ±∞ )` returns 1.
- `pow( +1, y )` returns 1 for any y , even a NaN.
- `pow( x, ±0 )` returns 1 for any x , even a NaN.
- `pow( x, y )` returns a NaN for finite $x < 0$ and finite non-integer y .
- `pow( x, -∞ )` returns $+∞$ for $|x| < 1$.
- `pow( x, -∞ )` returns $+0$ for $|x| > 1$.
- `pow( x, +∞ )` returns $+0$ for $|x| < 1$.
- `pow( x, +∞ )` returns $+∞$ for $|x| > 1$.
- `pow( -∞, y )` returns -0 for y an odd integer < 0 .
- `pow( -∞, y )` returns $+0$ for $y < 0$ and not an odd integer.
- `pow( -∞, y )` returns $-∞$ for y an odd integer > 0 .
- `pow( -∞, y )` returns $+∞$ for $y > 0$ and not an odd integer.
- `pow( +∞, y )` returns $+0$ for $y < 0$.
- `pow( +∞, y )` returns $+∞$ for $y > 0$.

• **OpExtInst round:**

- `round( ±0 )` returns $±0$.
- `round( ±∞ )` returns $±∞$.

• **OpExtInst sin:**

- `sin( ±0 )` returns $±0$.
- `sin( ±∞ )` returns a NaN.

• **OpExtInst sinh:**

- `sinh( ±0 )` returns $±0$.
- `sinh( ±∞ )` returns $±∞$.

• **OpExtInst tan:**

- `tan( ±0 )` returns $±0$.
- `tan( ±∞ )` returns a NaN.

• **OpExtInst tanh:**

- $\tanh(\pm 0)$ returns ± 0 .
- $\tanh(\pm \infty)$ returns ± 1 .
- **OpExtInst tgamma:**
 - $\text{tgamma}(\pm 0)$ returns $\pm \infty$.
 - $\text{tgamma}(x)$ returns a NaN for x a negative integer.
 - $\text{tgamma}(-\infty)$ returns a NaN.
 - $\text{tgamma}(+\infty)$ returns $+\infty$.
- **OpExtInst trunc:**
 - $\text{trunc}(\pm 0)$ returns ± 0 .
 - $\text{trunc}(\pm \infty)$ returns $\pm \infty$.

6.7.1. Additional Requirements Beyond ISO/IEC 9899:TC2

All functions that return a NaN should return a quiet NaN.

- **OpExtInst acospi:**
 - $\text{acospi}(1)$ returns $+0$.
 - $\text{acospi}(x)$ returns a NaN for $|x| > 1$.
- **OpExtInst asinpi:**
 - $\text{asinpi}(\pm 0)$ returns ± 0 .
 - $\text{asinpi}(x)$ returns a NaN for $|x| > 1$.
- **OpExtInst atanpi:**
 - $\text{atanpi}(\pm 0)$ returns ± 0 .
 - $\text{atanpi}(\pm \infty)$ returns ± 0.5 .
- **OpExtInst atan2pi:**
 - $\text{atan2pi}(\pm 0, -0)$ returns ± 1 .
 - $\text{atan2pi}(\pm 0, +0)$ returns ± 0 .
 - $\text{atan2pi}(\pm 0, x)$ returns ± 1 for $x < 0$.
 - $\text{atan2pi}(\pm 0, x)$ returns ± 0 for $x > 0$.
 - $\text{atan2pi}(y, \pm 0)$ returns -0.5 for $y < 0$.
 - $\text{atan2pi}(y, \pm 0)$ returns 0.5 for $y > 0$.
 - $\text{atan2pi}(\pm y, -\infty)$ returns ± 1 for finite $y > 0$.
 - $\text{atan2pi}(\pm y, +\infty)$ returns ± 0 for finite $y > 0$.
 - $\text{atan2pi}(\pm \infty, x)$ returns ± 0.5 for finite x .
 - $\text{atan2pi}(\pm \infty, -\infty)$ returns ± 0.75 .
 - $\text{atan2pi}(\pm \infty, +\infty)$ returns ± 0.25 .
- **OpExtInst ceil:**

- `ceil( -1 < x < 0 )` returns -0.
- **OpExtInst cospi:**
 - `cospi( ±0 )` returns 1.
 - `cospi( n + 0.5 )` returns +0 for any integer n where n + 0.5 is representable.
 - `cospi( ±∞ )` returns a NaN.
- **OpExtInst exp10:**
 - `exp10( ±0 )` returns 1.
 - `exp10( -∞ )` returns +0.
 - `exp10( +∞ )` returns +∞.
- **OpExtInst distance:**
 - `distance( x, y )` calculates the distance from x to y without overflow or extraordinary precision loss due to underflow.
- **OpExtInst fdim:**
 - `fdim( any, NaN )` returns a NaN.
 - `fdim( NaN, any )` returns a NaN.
- **OpExtInst fmod:**
 - `fmod( ±0, NaN )` returns a NaN.
- **OpExtInst fract:**
 - `fract( x, iptr )` shall not return a value greater than or equal to 1.0, and shall not return a value less than 0.
 - `fract( +0, iptr )` returns +0 and +0 in iptr.
 - `fract( -0, iptr )` returns -0 and -0 in iptr.
 - `fract( +inf, iptr )` returns +0 and +inf in iptr.
 - `fract( -inf, iptr )` returns -0 and -inf in iptr.
 - `fract( NaN, iptr )` returns a NaN and a NaN in iptr.
- **OpExtInst frexp:**
 - `frexp( ±∞, exp )` returns ±∞ and stores 0 in exp.
 - `frexp( NaN, exp )` returns a NaN and stores 0 in exp.
- **OpExtInst length:**
 - `length` calculates the length of a vector without overflow or extraordinary precision loss due to underflow.
- **OpExtInst lgamma_r:**
 - `lgamma_r( x, signp )` returns 0 in signp if x is zero or a negative integer.
- **OpExtInst nextafter:**
 - `nextafter( -0, y > 0 )` returns smallest positive denormal value.

- `nextafter( +0, y < 0 )` returns smallest negative denormal value.

- **OpExtInst normalize:**

- `normalize` shall reduce the vector to unit length, pointing in the same direction without overflow or extraordinary precision loss due to underflow.
- `normalize( v )` returns `v` if all elements of `v` are zero.
- `normalize( v )` returns a vector full of NaNs if any element is a NaN.
- `normalize( v )` for which any element in `v` is infinite shall proceed as if the elements in `v` were replaced as follows:

```
for( i = 0; i < sizeof(v) / sizeof(v[0] ); i++ )
    v[i] = isinf(v[i] ) ? copysign(1.0, v[i]) : 0.0 * v [i];
```

- **OpExtInst pow:**

- `pow( ±0, -∞ )` returns $+\infty$

- **OpExtInst pown:**

- `pown( x, 0 )` returns 1 for any `x`, even zero, NaN or infinity.
- `pown( ±0, n )` returns $\pm\infty$ for odd `n` < 0.
- `pown( ±0, n )` returns $+\infty$ for even `n` < 0.
- `pown( ±0, n )` returns $+0$ for even `n` > 0.
- `pown( ±0, n )` returns ± 0 for odd `n` > 0.

- **OpExtInst powr:**

- `powr( x, ±0 )` returns 1 for finite `x` > 0.
- `powr( ±0, y )` returns $+\infty$ for finite `y` < 0.
- `powr( ±0, -∞ )` returns $+\infty$.
- `powr( ±0, y )` returns $+0$ for `y` > 0.
- `powr( +1, y )` returns 1 for finite `y`.
- `powr( x, y )` returns a NaN for `x` < 0.
- `powr( ±0, ±0 )` returns a NaN.
- `powr( +∞, ±0 )` returns a NaN.
- `powr( +1, ±∞ )` returns a NaN.
- `powr( x, NaN )` returns a NaN for `x` >= 0.
- `powr( NaN, y )` returns a NaN.

- **OpExtInst rint:**

- `rint( -0.5 <= x < 0 )` returns -0.

- **OpExtInst remquo:**

- `remquo(x, y, &quo)` returns a NaN and 0 in `quo` if `x` is $\pm\infty$, or if `y` is 0 and the other argument

is non-NaN or if either argument is a NaN.

- **OpExtInst rootn:**

- `rootn( ±0, n )` returns $\pm\infty$ for odd $n < 0$.
- `rootn( ±0, n )` returns $+\infty$ for even $n < 0$.
- `rootn( ±0, n )` returns $+0$ for even $n > 0$.
- `rootn( ±0, n )` returns ± 0 for odd $n > 0$.
- `rootn( x, n )` returns a NaN for $x < 0$ and n is even.
- `rootn( x, 0 )` returns a NaN.

- **OpExtInst round:**

- `round(  $-0.5 < x < 0$  )` returns -0 .

- **OpExtInst sinpi:**

- `sinpi( ±0 )` returns ± 0 .
- `sinpi( +n )` returns $+0$ for positive integers n .
- `sinpi( -n )` returns -0 for negative integers n .
- `sinpi(  $\pm\infty$  )` returns a NaN.

- **OpExtInst tanpi:**

- `tanpi( ±0 )` returns ± 0 .
- `tanpi(  $\pm\infty$  )` returns a NaN.
- `tanpi( n )` returns `copysign( 0.0, n )` for even integers n .
- `tanpi( n )` returns `copysign( 0.0, -n )` for odd integers n .
- `tanpi(  $n + 0.5$  )` returns $+\infty$ for even integer n where $n + 0.5$ is representable.
- `tanpi(  $n + 0.5$  )` returns $-\infty$ for odd integer n where $n + 0.5$ is representable.

- **OpExtInst trunc:**

- `trunc(  $-1 < x < 0$  )` returns -0 .

6.7.2. Changes to ISO/IEC 9899:TC2 Behavior

OpExtInst modf behaves as though implemented by:

```
gentype modf( gentype value, gentype *iptr )
{
    *iptr = trunc( value );
    return copysign( isinf( value ) ? 0.0 : value - *iptr, value );
}
```

OpExtInst rint always rounds according to round to nearest even rounding mode even if the caller is in some other rounding mode.

6.7.3. Edge Case Behavior in Flush To Zero Mode

If denormals are flushed to zero, then a function may return one of four results:

1. Any conforming result for non-flush-to-zero mode.
2. If the result given by 1 is a sub-normal before rounding, it may be flushed to zero.
3. Any non-flushed conforming result for the function if one or more of its sub-normal operands are flushed to zero.
4. If the result of 3 is a sub-normal before rounding, the result may be flushed to zero.

In each of the above cases, if an operand or result is flushed to zero, the sign of the zero is undefined.

If subnormals are flushed to zero, a device may choose to conform to the following edge cases for **OpExtInst nextafter** instead of those listed in [Additional Requirements Beyond ISO/IEC 9899:TC2 section](#):

- `nextafter( +smallest normal, y < +smallest normal )` returns `+0`.
- `nextafter( -smallest normal, y > -smallest normal )` returns `-0`.
- `nextafter( -0, y > 0 )` returns smallest positive normal value.
- `nextafter( +0, y < 0 )` returns smallest negative normal value.

For clarity, subnormals or denormals are defined to be the set of representable numbers in the range $0 < x < \text{TYPE_MIN}$ and $-\text{TYPE_MIN} < x < -0$. They do not include ± 0 . A non-zero number is said to be sub-normal before rounding if, after normalization, its radix-2 exponent is less than $(\text{TYPE_MIN_EXP} - 1)$.^[1]

[1] Here `TYPE_MIN` and `TYPE_MIN_EXP` should be substituted by constants appropriate to the floating-point type under consideration, such as `FLT_MIN` and `FLT_MIN_EXP` for float.

Chapter 7. Image Addressing and Filtering

This section describes how image operations behave in an OpenCL environment.

7.1. Image Coordinates

Let w_t , h_t and d_t be the width, height (or image array size for a 1D image array) and depth (or image array size for a 2D image array) of the image in pixels. Let coord.xy (also referred to as (s, t)) or coord.xyz (also referred to as (s, t, r)) be the coordinates specified to an image read instruction (such as **OpImageRead**) or an image write instruction (such as **OpImageWrite**).

If image coordinates specified to an image read instruction are normalized (as specified in the sampler), the s , t , and r coordinate values are multiplied by w_t , h_t and d_t respectively to generate the unnormalized coordinate values. For image arrays, the image array coordinate (i.e. t if it is a 1D image array or r if it is a 2D image array) specified to the image read instruction must always be the unnormalized image coordinate value.

Image coordinates specified to an image write instruction are always unnormalized image coordinate values.

Let (u, v, w) represent the unnormalized image coordinate values.

If values in (s, t, r) or (u, v, w) are INF or NaN, the behavior of the image read instruction or image write instruction is undefined.

7.2. Addressing and Filter Modes

After generating the image coordinate (u, v, w) we apply the appropriate addressing and filter mode to generate the appropriate sample locations to read from the image.

7.2.1. Clamp and None Addressing Modes

We first describe how the addressing and filter modes are applied to generate the appropriate sample locations to read from the image if the addressing mode is **CL_ADDRESS_CLAMP**, **CL_ADDRESS_CLAMP_TO_EDGE**, or **CL_ADDRESS_NONE**.

7.2.1.1. Nearest Filtering

When the filter mode is **CL_FILTER_NEAREST**, the result of the image read instruction is the image element that is nearest (in Manhattan distance) to the image element location (i, j, k) . The image element location (i, j, k) is computed as:

$$\begin{aligned} i &= \text{address_mode}((\text{int})\text{floor}(u)) \\ j &= \text{address_mode}((\text{int})\text{floor}(v)) \\ k &= \text{address_mode}((\text{int})\text{floor}(w)) \end{aligned}$$

For a 3D image, the image element at location (i, j, k) becomes the color value. For a 2D image, the image element at location (i, j) becomes the color value.

The below table describes the `address_mode` function.

Table 8. Addressing Modes to Generate Texel Location

Addressing Mode	Result of <code>address_mode(coord)</code>
<code>CL_ADDRESS_CLAMP</code>	<code>clamp(coord, -1, size)</code>
<code>CL_ADDRESS_CLAMP_TO_EDGE</code>	<code>clamp(coord, 0, size - 1)</code>
<code>CL_ADDRESS_NONE</code>	<code>coord</code>

The size term in the table above is w_t for u, h_t for v and d_t for w.

The clamp function used in the table above is defined as:

$$\text{clamp}(a, b, c) = \text{return}(a < b) ? b : ((a > c) ? c : a)$$

If the addressing mode is `CL_ADDRESS_CLAMP` or `CL_ADDRESS_CLAMP_TO_EDGE`, and the selected texel location (i, j, k) refers to a location outside the image, the border color is used as the color value for the texel.

Otherwise, if the addressing mode is `CL_ADDRESS_NONE` and the selected texel location (i, j, k) refers to a location outside the image, the color value for the texel is undefined.

7.2.1.2. Linear Filtering

When the filter mode is `CL_FILTER_LINEAR`, a 2 x 2 square of image elements (for a 2D image) or a 2 x 2 x 2 cube of image elements (for a 3D image is selected). This 2 x 2 square or 2 x 2 x 2 cube is obtained as follows.

Let:

$$\begin{aligned} i0 &= \text{address_mode}((\text{int})\text{floor}(u - 0.5)) \\ j0 &= \text{address_mode}((\text{int})\text{floor}(v - 0.5)) \\ k0 &= \text{address_mode}((\text{int})\text{floor}(w - 0.5)) \\ i1 &= \text{address_mode}((\text{int})\text{floor}(u - 0.5) + 1) \\ j1 &= \text{address_mode}((\text{int})\text{floor}(v - 0.5) + 1) \\ k1 &= \text{address_mode}((\text{int})\text{floor}(w - 0.5) + 1) \\ a &= \text{frac}(u - 0.5) \\ b &= \text{frac}(v - 0.5) \\ c &= \text{frac}(w - 0.5) \end{aligned}$$

The frac function determines the fractional part of x and is computed as:

$$\text{frac}(x) = x - \text{floor}(x)$$

For a 3D image, the color value is computed as:

$$\begin{aligned}
T = & (1 - a) \times (1 - b) \times (1 - c) \times T_{i0j0k0} \\
& + a \times (1 - b) \times (1 - c) \times T_{i1j0k0} \\
& + (1 - a) \times b \times (1 - c) \times T_{i0j1k0} \\
& + a \times b \times (1 - c) \times T_{i1j1k0} \\
& + (1 - a) \times (1 - b) \times c \times T_{i0j0k1} \\
& + a \times (1 - b) \times c \times T_{i1j0k1} \\
& + (1 - a) \times b \times c \times T_{i0j1k1} \\
& + a \times b \times c \times T_{i1j1k1}
\end{aligned}$$

where T_{ijk} is the image element at location (i, j, k) in the 3D image.

For a 2D image, the color value is computed as:

$$\begin{aligned}
T = & (1 - a) \times (1 - b) \times T_{i0j0} \\
& + a \times (1 - b) \times T_{i1j0} \\
& + (1 - a) \times b \times T_{i0j1} \\
& + a \times b \times T_{i1j1}
\end{aligned}$$

where T_{ij} is the image element at location (i, j) in the 2D image.

If the addressing mode is `CL_ADDRESS_CLAMP` or `CL_ADDRESS_CLAMP_TO_EDGE`, and any of the selected T_{ijk} or T_{ij} refers to a location outside the image, the border color is used as the image element.

Otherwise, if the addressing mode is `CL_ADDRESS_NONE`, and any of the selected T_{ijk} or T_{ij} refers to a location outside the image, the color value is undefined.

If the image channel type is `CL_FLOAT` or `CL_HALF_FLOAT`, and any of the image elements T_{ijk} or T_{ij} is INF or NaN, the color value is undefined.

7.2.2. Repeat Addressing Mode

We now discuss how the addressing and filter modes are applied to generate the appropriate sample locations to read from the image if the addressing mode is `CL_ADDRESS_REPEAT`.

7.2.2.1. Nearest Filtering

When filter mode is `CL_FILTER_NEAREST`, the result of the image read instruction is the image element that is nearest (in Manhattan distance) to the image element location (i, j, k) . The image element location (i, j, k) is computed as:

$$\begin{aligned}
u &= (s - \text{floor}(s)) \times w_t \\
i &= (\text{int})\text{floor}(u) \\
\text{if } (i > w_t - 1) \\
&\quad i = i - w_t \\
v &= (t - \text{floor}(t)) \times h_t \\
j &= (\text{int})\text{floor}(v) \\
\text{if } (j > h_t - 1) \\
&\quad j = j - h_t \\
w &= (r - \text{floor}(r)) \times d_t \\
k &= (\text{int})\text{floor}(w) \\
\text{if } (k > d_t - 1) \\
&\quad k = k - d_t
\end{aligned}$$

For a 3D image, the image element at location (i, j, k) becomes the color value. For a 2D image, the image element at location (i, j) becomes the color value.

7.2.2.2. Linear Filtering

When filter mode is **CL_FILTER_LINEAR**, a 2 x 2 square of image elements for a 2D image or a 2 x 2 x 2 cube of image elements for a 3D image is selected. This 2 x 2 square or 2 x 2 x 2 cube is obtained as follows.

Let

```

u = (s - floor(s)) × wt
i0 = (int)floor(u - 0.5)
i1 = i0 + 1
if(i0 < 0)
i0 = wt + i0
if(i1 > wt - 1)
    i1 = i1 - wt
v = (t - floor(t)) × ht
j0 = (int)floor(v - 0.5)
j1 = j0 + 1
if(j0 < 0)
j0 = ht + j0
if(j1 > ht - 1)
    j1 = j1 - ht
w = (r - floor(r)) × dt
k0 = (int)floor(w - 0.5)
k1 = k0 + 1
if(k0 < 0)
    k0 = dt + k0
if(k1 > dt - 1)
    k1 = k1 - dt
a = frac(u - 0.5)
b = frac(v - 0.5)
c = frac(w - 0.5)

```

For a 3D image, the color value is computed as:

$$\begin{aligned}
 T = & (1 - a) \times (1 - b) \times (1 - c) \times T_{i0j0k0} \\
 & + a \times (1 - b) \times (1 - c) \times T_{i1j0k0} \\
 & + (1 - a) \times b \times (1 - c) \times T_{i0j1k0} \\
 & + a \times b \times (1 - c) \times T_{i1j1k0} \\
 & + (1 - a) \times (1 - b) \times c \times T_{i0j0k1} \\
 & + a \times (1 - b) \times c \times T_{i1j0k1} \\
 & + (1 - a) \times b \times c \times T_{i0j1k1} \\
 & + a \times b \times c \times T_{i1j1k1}
 \end{aligned}$$

where T_{ijk} is the image element at location (i, j, k) in the 3D image.

For a 2D image, the color value is computed as:

$$\begin{aligned}
 T = & (1 - a) \times (1 - b) \times T_{i0j0} \\
 & + a \times (1 - b) \times T_{i1j0} \\
 & + (1 - a) \times b \times T_{i0j1} \\
 & + a \times b \times T_{i1j1}
 \end{aligned}$$

where T_{ij} is the image element at location (i, j) in the 2D image.

If the image channel type is `CL_FLOAT` or `CL_HALF_FLOAT`, and any of the image elements T_{ijk} or T_{ij} is INF or NaN, the color value is undefined.

7.2.3. Mirrored Repeat Addressing Mode

We now discuss how the addressing and filter modes are applied to generate the appropriate sample locations to read from the image if the addressing mode is `CL_ADDRESS_MIRRORED_REPEAT`. The `CL_ADDRESS_MIRRORED_REPEAT` addressing mode causes the image to be read as if it is tiled at every integer seam, with the interpretation of the image data flipped at each integer crossing.

7.2.3.1. Nearest Filtering

When filter mode is `CL_FILTER_NEAREST`, the result of the image read instruction is the image element that is nearest (in Manhattan distance) to the image element location (i, j, k) . The image element location (i, j, k) is computed as:

$$\begin{aligned} s' &= 2.0f \times \text{rint}(0.5f \times s) \\ s' &= fabs(s - s') \\ u &= s' \times w_t \\ i &= (\text{int})\text{floor}(u) \\ i &= \min(i, w_t - 1) \\ t' &= 2.0f \times \text{rint}(0.5f \times t) \\ t' &= fabs(t - t') \\ v &= t' \times h_t \\ j &= (\text{int})\text{floor}(v) \\ j &= \min(j, h_t - 1) \\ r' &= 2.0f \times \text{rint}(0.5f \times r) \\ r' &= fabs(r - r') \\ w &= r' \times d_t \\ k &= (\text{int})\text{floor}(w) \\ k &= \min(k, d_t - 1) \end{aligned}$$

For a 3D image, the image element at location (i, j, k) becomes the color value. For a 2D image, the image element at location (i, j) becomes the color value.

7.2.3.2. Linear Filtering

When filter mode is `CL_FILTER_LINEAR`, a 2 x 2 square of image elements for a 2D image or a 2 x 2 x 2 cube of image elements for a 3D image is selected. This 2 x 2 square or 2 x 2 x 2 cube is obtained as follows.

Let

$$\begin{aligned}
s' &= 2.0f \times \text{rint}(0.5f \times s) \\
s' &= f\text{abs}(s - s') \\
u &= s' \times w_t \\
i0 &= (\text{int})\text{floor}(u - 0.5f) \\
i1 &= i0 + 1 \\
i0 &= \max(i0, 0) \\
i1 &= \min(i1, w_t - 1) \\
t' &= 2.0f \times \text{rint}(0.5f \times t) \\
t' &= f\text{abs}(t - t') \\
v &= t' \times h_t \\
j0 &= (\text{int})\text{floor}(v - 0.5f) \\
j1 &= j0 + 1 \\
j0 &= \max(j0, 0) \\
j1 &= \min(j1, h_t - 1) \\
r' &= 2.0f \times \text{rint}(0.5f \times r) \\
r' &= f\text{abs}(r - r') \\
w &= r' \times d_t \\
k0 &= (\text{int})\text{floor}(w - 0.5f) \\
k1 &= k0 + 1 \\
k0 &= \max(k0, 0) \\
k1 &= \min(k1, d_t - 1) \\
a &= \text{frac}(u - 0.5) \\
b &= \text{frac}(v - 0.5) \\
c &= \text{frac}(w - 0.5)
\end{aligned}$$

For a 3D image, the color value is computed as:

$$\begin{aligned}
T &= (1 - a) \times (1 - b) \times (1 - c) \times T_{i0j0k0} \\
&+ a \times (1 - b) \times (1 - c) \times T_{i1j0k0} \\
&+ (1 - a) \times b \times (1 - c) \times T_{i0j1k0} \\
&+ a \times b \times (1 - c) \times T_{i1j1k0} \\
&+ (1 - a) \times (1 - b) \times c \times T_{i0j0k1} \\
&+ a \times (1 - b) \times c \times T_{i1j0k1} \\
&+ (1 - a) \times b \times c \times T_{i0j1k1} \\
&+ a \times b \times c \times T_{i1j1k1}
\end{aligned}$$

where T_{ijk} is the image element at location (i, j, k) in the 3D image.

For a 2D image, the color value is computed as:

$$\begin{aligned}
T &= (1 - a) \times (1 - b) \times T_{i0j0} \\
&+ a \times (1 - b) \times T_{i1j0} \\
&+ (1 - a) \times b \times T_{i0j1} \\
&+ a \times b \times T_{i1j1}
\end{aligned}$$

where T_{ij} is the image element at location (i, j) in the 2D image.

For a 1D image, the color value is computed as:

$$T = (1 - a) \times T_{i0} + a \times T_{i1}$$

where T_i is the image element at location (i) in the 1D image.

If the image channel type is `CL_FLOAT` or `CL_HALF_FLOAT` and any of the image elements T_{ijk} or T_{ij} is INF or NaN, the color value is undefined.

7.3. Precision of Addressing and Filter Modes

If the sampler is specified as using unnormalized coordinates (floating-point or integer coordinates), filter mode set to `CL_FILTER_NEAREST` and addressing mode set to one of the following modes - `CL_ADDRESS_CLAMP`, `CL_ADDRESS_CLAMP_TO_EDGE` or `CL_ADDRESS_NONE` - the location of the image element in the image given by (i, j, k) will be computed without any loss of precision.

For all other sampler combinations of normalized or unnormalized coordinates, filter modes, and addressing modes, the relative error or precision of the addressing mode calculations and the image filter operation are not defined. To ensure precision of image addressing and filter calculations across any OpenCL device for these sampler combinations, developers may unnormalize the image coordinate in the kernel, and then implement the linear filter in the kernel with appropriate read image instructions with a sampler that uses unnormalized coordinates, filter mode set to `CL_FILTER_NEAREST`, addressing mode set to `CL_ADDRESS_CLAMP`, `CL_ADDRESS_CLAMP_TO_EDGE` or `CL_ADDRESS_NONE`, and finally performing the interpolation of color values read from the image to generate the filtered color value.

7.4. Conversion Rules

In this section we discuss conversion rules that are applied when reading and writing images in a kernel.

7.4.1. Conversion Rules for Normalized Integer Channel Data Types

In this section we discuss converting normalized integer channel data types to half-precision and single-precision floating-point values and vice-versa.

7.4.1.1. Converting Normalized Integer Channel Data Types to Half Precision Floating-point Values

For images created with any of the following image channel data types, image read instructions will convert the channel values from an unsigned integer to normalized half precision floating-point values in the range $[0.0h \dots 1.0h]$.

- `CL_UNORM_INT8`
- `CL_UNORM_INT16`
- `CL_UNORM_INT10X6_EXT`
- `CL_UNORM_INT12X4_EXT`

- `CL_UNORM_INT14X2_EXT`

For images created with any of the following image channel data types, image read instructions will convert the channel values from a signed integer to normalized half precision floating-point values in the range [-1.0h ... 1.0h].

- `CL_SNORM_INT8`
- `CL_SNORM_INT16`

These conversions are performed as follows:

- `CL_UNORM_INT8` (8-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{255}\right)$$

- `CL_UNORM_INT_101010` (10-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{1023}\right)$$

- `CL_UNORM_INT16` (16-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{65535}\right)$$

- `CL_UNORM_INT10X6_EXT` (10-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{1023}\right)$$

- `CL_UNORM_INT12X4_EXT` (12-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{4095}\right)$$

- `CL_UNORM_INT14X2_EXT` (14-bit unsigned integer) → `half`

$$\text{normalized_half_value}(x) = \text{round_to_half}\left(\frac{x}{16383}\right)$$

- `CL_SNORM_INT8` (8-bit signed integer) → `half`

$$\text{normalized_half_value}(x) = \max(-1.0h, \text{round_to_half}\left(\frac{x}{127}\right))$$

- `CL_SNORM_INT16` (16-bit signed integer) → `half`

$$\text{normalized_half_value}(x) = \max(-1.0h, \text{round_to_half}\left(\frac{x}{32767}\right))$$

The precision of the above conversions is ≤ 1.5 ulp except for the following cases:

For `CL_UNORM_INT8`:

- 0 must convert to 0.0h, and

- 255 must convert to 1.0h

For `CL_UNORM_INT_101010`:

- 0 must convert to 0.0h, and
- 1023 must convert to 1.0h

For `CL_UNORM_INT16`:

- 0 must convert to 0.0h, and
- 65535 must convert to 1.0h

For `CL_UNORM_INT10X6_EXT`:

- 0 must convert to 0.0h, and
- 1023 must convert to 1.0h

For `CL_UNORM_INT12X4_EXT`:

- 0 must convert to 0.0h, and
- 4095 must convert to 1.0h

For `CL_UNORM_INT14X2_EXT`:

- 0 must convert to 0.0h, and
- 16383 must convert to 1.0h

For `CL_SNORM_INT8`:

- -128 and -127 must convert to -1.0h,
- 0 must convert to 0.0h, and
- 127 must convert to 1.0h

For `CL_SNORM_INT16`:

- -32768 and -32767 must convert to -1.0h,
- 0 must convert to 0.0h, and
- 32767 must convert to 1.0h

7.4.1.2. Converting Half Precision Floating-point Values to Normalized Integer Channel Data Types

For images created with any of the following image channel data types, image write instructions will convert the half precision floating-point color value to an unsigned integer.

- `CL_UNORM_INT8`
- `CL_UNORM_INT16`

- `CL_UNORM_INT10X6_EXT`
- `CL_UNORM_INT12X4_EXT`
- `CL_UNORM_INT14X2_EXT`

For images created with any of the following image channel data types, image write instructions will convert the half precision floating-point color value to a signed integer.

- `CL_SNORM_INT8`
- `CL_SNORM_INT16`

OpenCL implementations may choose to approximate the rounding mode used in the conversions described below. When approximate rounding is used instead of the preferred rounding, the result of the conversion must satisfy the bound given below.

The conversions from half precision floating-point values to normalized integer values are performed as follows:

- `half` → `CL_UNORM_INT8` (8-bit unsigned integer)

$$f(x) = \max(0, \min(255, 255 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- `half` → `CL_UNORM_INT16` (16-bit unsigned integer)

$$f(x) = \max(0, \min(65535, 65535 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- `half` → `CL_UNORM_INT10X6_EXT` (10-bit unsigned integer)

$$f(x) = \max(0, \min(1023, 1023 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint10}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint10}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- **half** → **CL_UNORM_INT12X4_EXT** (12-bit unsigned integer)

$$f(x) = \max(0, \min(4095, 4095 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint12}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint12}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- **half** → **CL_UNORM_INT14X2_EXT** (14-bit unsigned integer)

$$f(x) = \max(0, \min(16383, 16383 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint14}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint14}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- **half** → **CL_SNORM_INT8** (8-bit signed integer)

$$f(x) = \max(-128, \min(127, 127 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_int8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_int8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- **half** → **CL_SNORM_INT16** (16-bit signed integer)

$$f(x) = \max(-32768, \min(32767, 32767 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_int16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_int16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

7.4.1.3. Converting Normalized Integer Channel Data Types to Floating-point Values

For images created with any of the following image channel data types, image read instructions will convert the channel values from an unsigned integer to normalized floating-point values in the range [0.0f ... 1.0f].

- CL_UNORM_INT8
- CL_UNORM_INT16
- CL_UNORM_INT10X6_EXT
- CL_UNORM_INT12X4_EXT
- CL_UNORM_INT14X2_EXT

For images created with any of the following image channel data types, image read instructions will convert the channel values from a signed integer to normalized floating-point values in the range [-1.0f ... 1.0f].

- CL_SNORM_INT8
- CL_SNORM_INT16

These conversions are performed as follows:

- CL_UNORM_INT8 (8-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{255}\right)$$

- CL_UNORM_INT_101010 (10-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{1023}\right)$$

- CL_UNORM_INT16 (16-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{65535}\right)$$

- CL_UNORM_INT10X6_EXT (10-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{1023}\right)$$

- CL_UNORM_INT12X4_EXT (12-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{4095}\right)$$

- CL_UNORM_INT14X2_EXT (14-bit unsigned integer) → float

$$\text{normalized_float_value}(x) = \text{round_to_float}\left(\frac{x}{16383}\right)$$

- CL_SNORM_INT8 (8-bit signed integer) → float

$$\text{normalized_float_value}(x) = \max(-1.0f, \text{round_to_float}\left(\frac{x}{127}\right))$$

- CL_SNORM_INT16 (16-bit signed integer) → float

$$\text{normalized_float_value}(x) = \max(-1.0f, \text{round_to_float}\left(\frac{x}{32767}\right))$$

The precision of the above conversions is ≤ 1.5 ulp except for the following cases.

For `CL_UNORM_INT8`:

- 0 must convert to 0.0f, and
- 255 must convert to 1.0f

For `CL_UNORM_INT_101010`:

- 0 must convert to 0.0f, and
- 1023 must convert to 1.0f

For `CL_UNORM_INT16`:

- 0 must convert to 0.0f, and
- 65535 must convert to 1.0f

For `CL_UNORM_INT10X6_EXT`:

- 0 must convert to 0.0f, and
- 1023 must convert to 1.0f

For `CL_UNORM_INT12X4_EXT`:

- 0 must convert to 0.0f, and
- 4095 must convert to 1.0f

For `CL_UNORM_INT14X2_EXT`:

- 0 must convert to 0.0f, and
- 16383 must convert to 1.0f

For `CL_SNORM_INT8`:

- -128 and -127 must convert to -1.0f,
- 0 must convert to 0.0f, and
- 127 must convert to 1.0f

For `CL_SNORM_INT16`:

- -32768 and -32767 must convert to -1.0f,
- 0 must convert to 0.0f, and
- 32767 must convert to 1.0f

7.4.1.4. Converting Floating-point Values to Normalized Integer Channel Data Types

For images created with any of the following image channel data types, image write instructions will convert the floating-point color value to an unsigned integer.

- `CL_UNORM_INT8`
- `CL_UNORM_INT16`
- `CL_UNORM_INT10X6_EXT`
- `CL_UNORM_INT12X4_EXT`
- `CL_UNORM_INT14X2_EXT`

For images created with any of the following image channel data types, image write instructions will convert the floating-point color value to a signed integer.

- `CL_SNORM_INT8`
- `CL_SNORM_INT16`

OpenCL implementations may choose to approximate the rounding mode used in the conversions described below. When approximate rounding is used instead of the preferred rounding, the result of the conversion must satisfy the bound given below.

The conversions from single precision floating-point values to normalized integer values are performed as follows:

- `float` → `CL_UNORM_INT8` (8-bit unsigned integer)

$$f(x) = \max(0, \min(255, 255 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint8}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint8}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- `float` → `CL_UNORM_INT_101010` (10-bit unsigned integer)

$$f(x) = \max(0, \min(1023, 1023 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint10}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint10}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- `float` → `CL_UNORM_INT16` (16-bit unsigned integer)

$$f(x) = \max(0, \min(65535, 65535 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint16}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint16}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- **float** → **CL_UNORM_INT10X6_EXT** (10-bit unsigned integer)

$$f(x) = \max(0, \min(1023, 1023 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint10}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint10}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- **float** → **CL_UNORM_INT12X4_EXT** (12-bit unsigned integer)

$$f(x) = \max(0, \min(4095, 4095 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint12}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint12}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- **float** → **CL_UNORM_INT14X2_EXT** (14-bit unsigned integer)

$$f(x) = \max(0, \min(16383, 16383 \times x))$$

$$f_{\text{preferred}}(x) = \begin{cases} \text{round_to_nearest_even_uint14}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$f_{\text{approx}}(x) = \begin{cases} \text{round_to_impl_uint14}(f(x)) & x \neq \infty \text{ and } x \neq \text{NaN} \\ \text{implementation-defined} & x = \infty \text{ or } x = \text{NaN} \end{cases}$$

$$|f(x) - f_{\text{approx}}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq \text{NaN}$$

- **float** → **CL_SNORM_INT8** (8-bit signed integer)

$$f(x) = \max(-128, \min(127, 127 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint8}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

- **float** → **CL_SNORM_INT16** (16-bit signed integer)

$$f(x) = \max(-32768, \min(32767, 32767 \times x))$$

$$f_{preferred}(x) = \begin{cases} \text{round_to_nearest_even_uint16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$f_{approx}(x) = \begin{cases} \text{round_to_impl_uint16}(f(x)) & x \neq \infty \text{ and } x \neq NaN \\ \text{implementation-defined} & x = \infty \text{ or } x = NaN \end{cases}$$

$$|f(x) - f_{approx}(x)| \leq 0.6, x \neq \infty \text{ and } x \neq NaN$$

7.4.2. Conversion Rules for Half Precision Floating-point Channel Data Type

For images created with a channel data type of **CL_HALF_FLOAT**, the conversions of half to float and half to half are lossless. Conversions from float to half round the mantissa using the round to nearest even or round to zero rounding mode. Denormalized numbers for the half data type which may be generated when converting a float to a half may be flushed to zero. A float NaN must be converted to an appropriate NaN in the half type. A float INF must be converted to an appropriate INF in the half type.

7.4.3. Conversion Rules for Floating-point Channel Data Type

The following rules apply for reading and writing images created with channel data type of **CL_FLOAT**.

- NaNs may be converted to a NaN value(s) supported by the device.
- Denorms can be flushed to zero.
- All other values must be preserved.

7.4.4. Conversion Rules for Signed and Unsigned Integer Channel Data Types

For images created with any of the following image channel data types, image read instructions will return the integer values stored in the image at specified location, after sign extension to 32 bits for integer values smaller than 32 bits.

- **CL_SIGNED_INT8**
- **CL_SIGNED_INT16**

- `CL_SIGNED_INT32`

Likewise, for images created with any of the following image channel data types, image read instructions will return the integer values stored in the image at specified location, after zero extension to 32 bits for integer values smaller than 32 bits.

- `CL_UNSIGNED_INT8`
- `CL_UNSIGNED_INT16`
- `CL_UNSIGNED_INT32`
- `CL_UNSIGNED_INT10X6_EXT`
- `CL_UNSIGNED_INT12X4_EXT`
- `CL_UNSIGNED_INT14X2_EXT`

Image write instructions will perform one of the following conversions:

- 32 bit signed integer → `CL_SIGNED_INT8` (8-bit signed integer):

$$\text{int8_value}(x) = \text{clamp}(x, -128, 127)$$

- 32 bit signed integer → `CL_SIGNED_INT16` (16-bit signed integer):

$$\text{int16_value}(x) = \text{clamp}(x, -32768, 32767)$$

- 32 bit signed integer → `CL_SIGNED_INT32` (32-bit signed integer):

$$\text{int32_value}(x) = x \quad (\text{no conversion})$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT8` (8-bit unsigned integer):

$$\text{uint8_value}(x) = \text{clamp}(x, 0, 255)$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT16` (16-bit unsigned integer):

$$\text{uint16_value}(x) = \text{clamp}(x, 0, 65535)$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT32` (32-bit unsigned integer):

$$\text{uint32_value}(x) = x \quad (\text{no conversion})$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT10X6_EXT` (10-bit unsigned integer):

$$\text{uint10_value}(x) = \text{clamp}(x, 0, 1023)$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT12X4_EXT` (12-bit unsigned integer):

$$\text{uint12_value}(x) = \text{clamp}(x, 0, 4095)$$

- 32 bit unsigned integer → `CL_UNSIGNED_INT14X2_EXT` (14-bit unsigned integer):

$$\text{uint14_value}(x) = \text{clamp}(x, 0, 16383)$$

The conversions described in this section must be correctly saturated.

7.4.5. Conversion Rules for sRGBA and sBGRA Images

Standard RGB data, which roughly displays colors in a linear ramp of luminosity levels such that an average observer, under average viewing conditions, can view them as perceptually equal steps on an average display. All 0s maps to 0.0f, and all 1s maps to 1.0f. The sequence of unsigned integer encodings between all 0s and all 1s represent a nonlinear progression in the floating-point interpretation of the numbers between 0.0f to 1.0f. For more detail, see the [SRGB color standard](#).

Conversion from sRGB space is automatically done the image read instruction if the image channel order is one of the sRGB values described above. When reading from an sRGB image, the conversion from sRGB to linear RGB is performed before filtering is applied. If the format has an alpha channel, the alpha data is stored in linear color space. Conversion to sRGB space is automatically done by the image write instruction if the image channel order is one of the sRGB values described above and the device supports writing to sRGB images.

If the format has an alpha channel, the alpha data is stored in linear color space.

1. The following process is used by image read instructions to convert a normalized 8-bit unsigned integer sRGB color value x to a floating-point linear RGB color value y :

1. Convert a normalized 8-bit unsigned integer sRGB value x to a floating-point sRGB value r as per rules described in [Converting Normalized Integer Channel Data Types to Floating-point Values](#) section.

$$r = \text{normalized_float_value}(x)$$

2. Convert a floating-point sRGB value r to a floating-point linear RGB color value y :

$$c_{linear}(x) = \begin{cases} \frac{r}{12.92} & r \geq 0 \text{ and } r \leq 0.04045 \\ (\frac{r + 0.055}{1.055})^{2.4} & r > 0.04045 \text{ and } \leq 1 \end{cases}$$
$$y = c_{linear}(r)$$

2. The following process is used by image write instructions to convert a linear RGB floating-point color value y to a normalized 8-bit unsigned integer sRGB value x :

1. Convert a floating-point linear RGB value y to a normalized floating-point sRGB value r :

$$c_{linear}(x) = \begin{cases} 0 & y \geq NaN \text{ or } y < 0 \\ 12.92 \times y & y \geq 0 \text{ and } y < 0.0031308 \\ 1.055 \times y^{(\frac{1}{2.4})} & y \geq 0.0031308 \text{ and } y \leq 1 \\ 1 & y > 1 \end{cases}$$
$$r = c_{sRGB}(y)$$

2. Convert a normalized floating-point sRGB value r to a normalized 8-bit unsigned integer sRGB value x as per rules described in [Converting Floating-point Values to Normalized Integer Channel Data Types](#) section.

$$g(r) = \begin{cases} f_{preferred}(r) & \text{if rounding mode is round to even} \\ f_{approx}(r) & \text{if implementation-defined rounding mode} \end{cases}$$

$$x = g(r)$$

The accuracy required when converting a normalized 8-bit unsigned integer sRGB color value x to a floating-point linear RGB color value y is given by:

$$|x - 255 \times c_{sRGB}(y)| \leq 0.5$$

The accuracy required when converting a linear RGB floating-point color value y to a normalized 8-bit unsigned integer sRGB value x is given by:

$$|x - 255 \times c_{sRGB}(y)| \leq 0.6$$

7.5. Selecting an Image from an Image Array

Let (u, v, w) represent the unnormalized image coordinate values for reading from and/or writing to a 2D image in a 2D image array.

When read using a sampler, the 2D image layer selected is computed as:

$$layer = clamp(rint(w), 0, d_t - 1)$$

otherwise the layer selected is computed as:

$$layer = w$$

(since w is already an integer) and the result is undefined if w is not one of the integers $0, 1, \dots, d_t - 1$.

Let (u, v) represent the unnormalized image coordinate values for reading from and/or writing to a 1D image in a 1D image array.

When read using a sampler, the 1D image layer selected is computed as:

$$layer = clamp(rint(v), 0, h_t - 1)$$

otherwise the layer selected is computed as:

$$layer = v$$

(since v is already an integer) and the result is undefined if v is not one of the integers $0, 1, \dots, h_t - 1$.

7.6. Coordinate Format for Reading and Writing Images

This section describes supported types for the *Coordinate* operand used by image read instructions (such as **OpImageRead**) or image write instructions (such as **OpImageWrite**). The supported operand types are determined by the **OpImageType** for the image and whether the image is being read from or written to.

When reading from an image of the following image types, behavior is undefined unless the *Coordinate* operand is one of the supported coordinate types:

Table 9. Mapping Image Types to Coordinate Types for Reading

<i>Dim</i>	<i>Depth</i>	<i>Arrayed</i>	Supported Coordinate Type
1D	0	0	OpTypeInt with <i>Width</i> equal to 32 OpTypeFloat with <i>Width</i> equal to 32
1D	0	1	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 2 components of OpTypeFloat with <i>Width</i> equal to 32 The array index is provided by the second component of the <i>Coordinate</i> .
2D	0	0	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 2 components of OpTypeFloat with <i>Width</i> equal to 32
2D	1	0	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 2 components of OpTypeFloat with <i>Width</i> equal to 32
2D	0	1	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 4 components of OpTypeFloat with <i>Width</i> equal to 32 The array index is provided by the third component of the <i>Coordinate</i> . The fourth component of the <i>Coordinate</i> is ignored.
2D	1	1	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 4 components of OpTypeFloat with <i>Width</i> equal to 32 The array index is provided by the third component of the <i>Coordinate</i> . The fourth component of the <i>Coordinate</i> is ignored.
3D	0	0	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 OpTypeVector with 4 components of OpTypeFloat with <i>Width</i> equal to 32 The fourth component of the <i>Coordinate</i> is ignored.
Buffer	0	0	OpTypeInt with <i>Width</i> equal to 32 OpTypeFloat with <i>Width</i> equal to 32

For the integer types for the *Coordinate* operand, behavior is undefined unless the image is read without a sampler (such as **OpImageRead**), or with a sampler using non-normalized texel coordinates, **Nearest** filtering, and either the **None**, **ClampToEdge**, or **Clamp** addressing mode.

When writing to an image of the following image types, behavior is undefined unless the *Coordinate* operand is one of the supported coordinate types:

Table 10. Mapping Image Types to Coordinate Types for Writing

<i>Dim</i>	<i>Depth</i>	<i>Arrayed</i>	Supported Coordinate Type
1D	0	0	OpTypeInt with <i>Width</i> equal to 32
1D	0	1	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32 The array index is provided by the second component of the <i>Coordinate</i> .
2D	0	0	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32
2D	1	0	OpTypeVector with 2 components of OpTypeInt with <i>Width</i> equal to 32
2D	0	1	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 The array index is provided by the third component of the <i>Coordinate</i> . The fourth component of the <i>Coordinate</i> is ignored.
2D	1	1	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 The array index is provided by the third component of the <i>Coordinate</i> . The fourth component of the <i>Coordinate</i> is ignored.
3D	0	0	OpTypeVector with 4 components of OpTypeInt with <i>Width</i> equal to 32 The fourth component of the <i>Coordinate</i> is ignored.
Buffer	0	0	OpTypeInt with <i>Width</i> equal to 32

7.7. Data Format for Reading and Writing Images

This section describes how image element data is returned by an image read instruction or passed as the *Texel* data that is written by an image write instruction:

Whether the image element data is a vector or a scalar depends on the image channel order. For the following image channel orders, the image element data is mapped to components of a vector, and behavior is undefined unless the image element data is a four component vector type:

Table 11. Mapping Image Data to Vector Components

Image Channel Order	Vector Components
R, Rx	(R, 0, 0, 1)
A	(0, 0, 0, A)
RG, RGx	(R, G, 0, 1)
RGB, RGBx, sRGB, sRGBx	(R, G, B, 1)
RGBA, BGRA, ARGB, ABGR, sRGBA, sBGRA	(R, G, B, A)
Intensity	(I, I, I, I)
Luminance	(L, L, L, 1)

For the following image channel orders, behavior is undefined unless the image element data is a scalar type:

Table 12. Scalar Image Data

Image Channel Order	Scalar Value
Depth	D
DepthStencil	D

The vector component data type or scalar data type is dependent on the image channel data type. For the following image channel data types, behavior is undefined unless the image element data type is the specified data type:

Table 13. Mapping Image Channel Data Types to Data Types

Image Channel Data Type	Data Type
SnormInt8, SnormInt16, UnormInt8, UnormInt16, UnormShort565, UnormShort555, UnormInt101010, UnormInt101010_2, UnormInt24, HalfFloat, Float Additionally if the <code>cl_ext_image_unorm_int_2_101010</code> extension is supported: UnormInt2_101010EXT Additionally if the <code>cl_ext_image_unsigned_10x6_12x4_14x2</code> extension is supported: UnormInt10X6EXT, UnormInt12X4EXT, UnormInt14X2EXT	OpTypeFloat , with <i>Width</i> equal to 16 or 32.

Image Channel Data Type	Data Type
SignedInt8, SignedInt16, SignedInt32, UnsignedInt8, UnsignedInt16, UnsignedInt32 Additionally if the <code>cl_ext_image_unsigned_10x6_12x4_14x2</code> extension is supported: UnsignedInt10X6EXT, UnsignedInt12X4EXT, UnsignedInt14X2EXT	OpTypeInt , with <i>Width</i> equal to 32.

For the image channel data types that return integer data, when reading from an image with a sampler, behavior is undefined unless the sampler is using **Nearest** filtering.

7.8. Sampled and Sampler-less Reads

SPIR-V instructions that read from an image without a sampler (such as **OpImageRead**) behave exactly the same as the corresponding image read instruction with a sampler that has *Sampler Filter Mode* set to **Nearest**, **Non-Normalized** coordinates, and *Sampler Addressing Mode* set to **None**.

There is one exception for cases where the image being read has *Image Format* equal to a floating-point type (such as **R32f**). In this exceptional case, when channel data values are denormalized, the non-sampler image read instruction may return the denormalized data, while the sampler image read instruction may flush denormalized channel data values to zero. The coordinates must be between 0 and image size in that dimension, non inclusive.

Chapter 8. Normative References

1. *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2008, <http://dx.doi.org/10.1109/IEEESTD.2008.4610935> , August, 2008.
2. “ISO/IEC 9899:1999 - Programming Languages - C”, with technical corrigenda TC1 and TC2, <https://www.iso.org/standard/29237.html> .
3. “ISO/IEC 14882:2014 - Information technology - Programming languages - C++”, <https://www.iso.org/standard/64029.html> .
4. “The OpenCL Specification, Version 3.0, Unified”, <https://www.khronos.org/registry/OpenCL/> .
5. “The OpenCL C Specification, Version 3.0”, <https://www.khronos.org/registry/OpenCL/> .
6. “The OpenCL C++ 1.0 Specification”, <https://www.khronos.org/registry/OpenCL/> .
7. “SPIR-V Specification, Version 1.6, Unified”, <https://www.khronos.org/registry/spir-v/> .
8. “OpenCL Extended Instruction Set Specification”, <https://www.khronos.org/registry/spir-v/> .
9. Jean-Michel Muller. *On the definition of $ulp(x)$* . RR-5504, INRIA. 2005, pp.16. <inria-00070503> Currently hosted at <https://hal.inria.fr/inria-00070503/document>.
10. “IEC 61966-2-1:1999 Multimedia systems and equipment - Colour measurement and management - Part 2-1: Colour management - Default RGB colour space - sRGB”, <https://webstore.iec.ch/publication/6169> .

Appendix A: Changes to OpenCL

Changes to the OpenCL SPIR-V Environment specifications between successive versions are summarized below.

Summary of changes to OpenCL 3.0

The first non-experimental version of the OpenCL 3.0 specifications was **v3.0.5**.

Changes from **v3.0.5**:

- Clarified sub-group barrier behavior in non-uniform control flow.
- Added required alignment of types.
- Added new extensions:
 - `cl_khr_subgroup_extended_types`
 - `cl_khr_subgroup_non_uniform_vote`
 - `cl_khr_subgroup_ballot`
 - `cl_khr_subgroup_non_uniform_arithmetic`
 - `cl_khr_subgroup_shuffle`
 - `cl_khr_subgroup_shuffle_relative`
 - `cl_khr_subgroup_clustered_reduce`

Changes from **v3.0.6**:

- Explicitly say that **OpTypeSampledImage** may be used in an OpenCL environment.
- Added the required type for SPIR-V built-in variables.
- Fixed several bugs and formatting in the fast math ULP tables.
- Added new extensions:
 - `cl_khr_extended_bit_ops`
 - `cl_khr_spirv_extended_debug_info`
 - `cl_khr_spirv_linkonce_odr`

Changes from **v3.0.8**:

- Clarified that some OpenCL `KHR` extensions also require SPIR-V extensions.

Changes from **v3.0.14**:

- Fixed several numerical compliance bugs, see [#937](#).

Changes from **v3.0.15**:

- Clarified that 16-bit and 64-bit floats can be passed as kernel arguments to SPIR-V kernels, see

[#1049](#).

- Adds the numerical value of the image channel order and image channel data type to several tables, see [#1050](#).
- Added new extension:
 - `cl_khr_kernel_clock` (experimental)

Changes from **v3.0.16**:

- Documented the error bounds for a non-derived `atan2` implementation with unsafe math optimizations, see [#1073](#).
- Documented supported `printf` operand types, see [#1236](#).
- Documented valid coordinate types when reading from or writing to images, see [#1242](#).

Changes from **v3.0.17**:

- Updated the ULP requirements for the half-precision divide and reciprocal, see [#1293](#).
- Clarified the unsafe math accuracy requirements for the embedded profile, see [#1318](#).
- Updated the SPIR-V environment for SPIR-V 1.3, 1.4, 1.5, and 1.6, see [#1331](#).

Changes from **v3.0.18**:

- Updated the ULP requirements for the half-precision square root, see [#1386](#).
- Updated the ULP requirements for the half-precision tangent, see [#1387](#).
- The following extension has been finalized and is no longer experimental:
 - `cl_khr_kernel_clock`
- Added new extension:
 - `cl_khr_spirv_queries`

Summary of Changes from OpenCL 3.0 to OpenCL 3.1

OpenCL 3.1 requires support for the SPIR-V 1.0, SPIR-V 1.1, SPIR-V 1.2, SPIR-V 1.3, and SPIR-V 1.4 intermediate languages.

Other changes in OpenCL 3.1:

- Added support for the `z` and `t` `printf` length specifiers, see [#654](#).
- Allowed optional image operands for image write instructions, see [#1451](#).
- Fixed a typo in the Image Data Types table header row, see [#1531](#).
- Promoted the following extensions to the core API:
 - `cl_khr_extended_bit_ops`
 - `cl_khr_integer_dot_product`
 - `cl_khr_subgroup_extended_types`

- `cl_khr_subgroup_rotate`
- `cl_khr_subgroup_shuffle`
- `cl_khr_subgroup_shuffle_relative`

There were no changes from **v3.1.0** to **v3.1.1**.

Changes from **v3.1.1** to **v3.1.2**:

- Documented more edge-case behavior directly vs. referencing the C99 specification, see [#1539](#).
- Fixed the image addressing algorithm rendering, see [#1570](#).
- Added a missing restriction when reading from an integer image with a sampler, see [#1594](#).